

强化学习在飞行控制中的应用：多平台综述

综述永动机

2026 年 3 月 19 日

摘要

强化学习 (Reinforcement Learning, RL) 作为一种数据驱动的智能决策方法,近年来在飞行控制领域取得了显著进展。本文系统综述了强化学习在多旋翼、固定翼和垂直起降 (VTOL) 三种主要飞行平台上的应用现状、技术挑战与发展趋势。首先,本文介绍了强化学习的基础理论,包括马尔可夫决策过程 [102]、值函数方法、策略梯度 [105] 方法和 Actor-Critic [93] 架构。随后,分别深入分析了强化学习在三种飞行平台上的具体应用,涵盖姿态控制、轨迹跟踪、特殊机动控制等核心任务。特别地,本文详细讨论了 Sim-to-Real [7] 迁移技术,这是将仿真训练策略成功部署到真实飞行系统的关键环节。最后,本文总结了当前面临的主要挑战,包括样本效率、安全性、实时性等问题,并展望了与大语言模型结合、神经辐射场应用等新兴技术趋势。

本文的主要贡献包括: (1) 系统梳理了 RL 在三种飞行平台上的应用现状,覆盖从底层姿态控制到高层任务规划的完整技术栈; (2) 深入分析了 Sim-to-Real 迁移的核心技术,包括域随机化 [7]、域适应 [88]、系统辨识 [97] 等方法; (3) 总结了当前面临的关键挑战,并提出了未来研究方向; (4) 提供了实用的算法超参数设置、仿真环境 [12] 配置和奖励函数设计模板,为研究人员提供参考。

关键词: 强化学习; 飞行控制; 多旋翼; 固定翼; VTOL; Sim-to-Real; 深度强化学习 [14]

1 引言

1.1 研究背景与动机

无人机技术的快速发展正在深刻改变着人类社会的多个领域。从消费级的航拍娱乐到工业级的物流配送,从农业植保到应急救援,无人机系统展现出前所未有的应用潜力。据统计,全球无人机市场规模预计将在 2025 年达到 428 亿美元,年复合增长率超过 15%。这一蓬勃发展的态势对飞行控制技术提出了更高的要求。

传统的飞行控制方法主要基于经典控制理论和现代控制理论。比例-积分-微分 (PID) 控制器因其结构简单、易于实现而被广泛应用于商业飞控系统。然而, PID 控制 [82] 器

在面对复杂非线性动力学、外部扰动和系统不确定性时，往往需要大量的人工调参，且难以保证最优性能。线性二次型调节器 (LQR) 和模型预测控制 [99](MPC) 等现代控制方法虽然能够处理更复杂的约束条件，但它们高度依赖于精确的系统模型。

强化学习作为一种数据驱动的机器学习方法，为飞行控制提供了全新的技术路径。与传统控制方法不同，强化学习通过与环境的交互来学习最优控制 [104] 策略，无需预先知道精确的系统模型。这一特性使得强化学习特别适合处理飞行控制中的非线性、不确定性和复杂约束问题。

2017 年 Sadeghi 和 Levine 提出的 CAD2RL [3] 方法首次实现了完全在仿真中训练、直接部署到真实无人机的视觉导航控制。2019 年，Kaufmann 等人在 IEEE Transactions on Robotics 上发表的 Deep Drone Racing [2] 工作，展示了强化学习训练的控制器能够达到人类世界冠军级别的竞速性能。2021 年，Loquercio 等人在 Nature [1] 上发表的研究更是证明了仅使用机载传感和计算，深度强化学习可以实现与人类飞行员相当的高速敏捷飞行能力。

图 1 展示了本文综述涵盖的三种主要飞行平台。多旋翼平台以其垂直起降和悬停能力成为最广泛应用的无人机类型；固定翼平台凭借其高效的前飞性能在长航程任务中具有优势；而垂直起降 (VTOL) 平台则结合了两者的优点，能够在狭小空间起降并实现高效巡航。

强化学习在飞行控制中的应用动机可以从以下几个方面理解：

首先，**模型不确定性**是飞行控制中的核心挑战。飞行器的气动特性受到多种因素影响，包括飞行速度、攻角、侧滑角、高度、大气条件等。建立覆盖整个飞行包线的精确模型极其困难。强化学习通过与环境的直接交互来学习控制策略，能够有效应对模型不确定性。

其次，**非线性动力学**使得传统线性控制方法难以在整个飞行包线内保持最优性能。飞行器的动力学本质上是高度非线性的，特别是在大攻角机动、高速飞行和过渡飞行等极端工况下。强化学习中的神经网络 [96] 函数逼近器能够自然地处理这种非线性映射关系。

第三，**复杂任务目标**往往需要多目标优化和长期决策。例如，在能量最优轨迹规划、编队协同控制和自主着陆等任务中，需要在多个相互冲突的目标之间进行权衡，并做出长时域的决策。强化学习的奖励机制能够灵活地编码这些复杂目标。

最后，**自适应能力**对于应对突发情况至关重要。飞行器在实际运行中可能遇到执行器故障、结构损伤、风切变等意外情况。强化学习训练的策略可以通过在线学习或元学习快速适应这些变化。

1.2 问题定义与挑战

飞行控制问题的核心可以形式化为：给定飞行器当前状态和目标状态，设计控制律生成执行器指令，使得飞行器能够安全、稳定、高效地达到目标状态。从强化学习的视角，这可以建模为一个马尔可夫决策过程 (MDP)，其中状态空间包含飞行器的位置、速

MULTIROTOR	FIXED-WING	VTOL (HYBRID)
		
KEY FEATURES: • Vertical Takeoff/Landing (VTOL) • Hover Capability • High Maneuverability	KEY FEATURES: • Efficient Long-Distance Flight • Extended Flight Time • High Payload Capacity	KEY FEATURES: • Combines VTOL/Hover + Efficient Cruise • No Runway Required • Versatile Mission Use
CONTROL CHALLENGES: • High Power Consumption • Limited Flight Time	CONTROL CHALLENGES: • Requires Runway for T/O/L • Cannot Hover • Low Maneuverability	CONTROL CHALLENGES: • Complex Control System • Increased Weight/Complexity • Higher Cost

图 1: 三种主要飞行平台对比：多旋翼、固定翼和 VTOL

度、姿态、角速度等信息，动作空间对应于电机转速、舵面偏转角等控制输入，奖励函数则综合考量跟踪精度、能耗、安全性等指标。

安全性约束：飞行控制的首要要求是安全性。飞行器必须始终保持在安全的飞行包线内，避免失速、结构过载、碰撞等危险情况。这些安全约束需要在 RL 的奖励函数或约束条件中显式编码，但这往往导致稀疏奖励问题，增加学习难度。

实时性要求：飞行控制系统通常需要数百赫兹的控制频率。深度神经网络的推理必须在严格的时间限制内完成，这对计算资源提出了挑战。特别是在资源受限的机载计算机上，需要在模型复杂度和推理速度之间取得平衡。

Sim-to-Real 鸿沟：仿真环境与真实环境之间存在动力学差异、感知差异和执行器差异。在仿真中训练的策略直接部署到真实系统时，性能往往会显著下降。如何缩小这一”仿真到现实鸿沟”是 RL 飞行控制实际应用的关键挑战。

样本效率：深度 RL 算法通常需要数百万甚至数十亿个时间步的训练数据。在真实飞行器上收集如此大量的数据成本高昂且风险巨大。提高样本效率，使算法能够从有限的数据中学习，是实际应用的必要条件。

可解释性与认证：神经网络控制器的决策过程难以解释，这与航空领域严格的认证要求 (如 DO-178C) 存在冲突。如何验证和认证基于 RL 的飞行控制系统，是阻碍其商业应用的重要障碍。

表 1: 三种飞行平台特性对比			
特性	多旋翼	固定翼	VTOL
起降方式	垂直起降	跑道起降	垂直起降
悬停能力	是	否	是
前飞效率	低	高	高
控制冗余度	高	低	高
动力学复杂度	中等	高	很高
主要应用场景	航拍、巡检	长航时侦察	物流配送
RL 研究成熟度	高	中	低

表 1对比了三种飞行平台的主要特性。可以看出，不同平台在起降方式、悬停能力、前飞效率等方面存在显著差异，这些差异直接影响了控制策略的设计。

多旋翼平台的控制挑战主要包括：(1) 欠驱动特性，即四旋翼只有 4 个控制输入 (4 个电机转速) 但需要控制 6 个自由度 (位置和姿态)；(2) 强耦合非线性，姿态控制与位置控制之间存在复杂的耦合关系；(3) 扰动敏感性，多旋翼质量轻、惯性小，容易受到风扰等外部干扰的影响；(4) 地面效应，在接近地面时旋翼产生的气流与地面的相互作用会显著改变气动特性。

固定翼平台的控制挑战主要包括：(1) 飞行包线约束，必须严格遵守失速速度、最大过载、最大攻角等约束条件；(2) 着陆复杂性，着陆过程涉及精确的高度、速度和姿

态控制, flare 机动等复杂操作; (3) 非最小相位特性, 某些操纵面输入会产生初始反向响应; (4) 参数变化, 随着燃油消耗和载荷变化, 飞行器质量特性会发生显著变化。

VTOL 平台的控制挑战最为复杂: (1) 模态转换控制, 在悬停模式、过渡模式和前飞模式之间切换时的稳定性保证; (2) 时变动力学, 转换过程中气动特性发生剧烈变化; (3) 控制分配, 多个执行器 (旋翼、舵面、推进器) 的协调控制; (4) 安全性要求, 模态转换失败可能导致灾难性后果。

除了上述平台特有的挑战, 强化学习在飞行控制中还面临一些**共性挑战**:

样本效率问题: 强化学习通常需要大量的交互样本来学习有效的策略。然而, 在真实飞行器上进行探索性学习成本高昂且风险巨大。如何在保证安全的前提下提高样本效率, 是实际应用中的关键问题。

安全性保证: 飞行控制对安全性有极高要求, 任何控制失误都可能导致飞行器坠毁。传统的强化学习算法在探索阶段可能采取危险动作, 这与飞行安全的要求相矛盾。安全强化学习 (Safe RL) 研究如何在满足安全约束的前提下进行学习。

Sim-to-Real 迁移: 由于真实世界训练的高成本和风险, 大多数强化学习研究在仿真环境中进行。然而, 仿真与现实之间存在动力学差异 (即“reality gap”), 导致在仿真中训练的策略在真实系统上性能下降。

实时性要求: 飞行控制系统通常需要数百赫兹的控制频率。深度神经网络的前向推理计算必须在严格的时间限制内完成, 这对计算资源有限的机载系统提出了挑战。

可解释性与认证: 神经网络控制器的决策过程往往难以解释, 这与航空领域严格的认证要求相冲突。如何验证和认证基于强化学习的飞行控制系统, 是其商业应用必须解决的问题。

1.3 研究范围与贡献

随着无人机技术的快速发展和强化学习算法的不断成熟, 将强化学习应用于飞行控制已成为学术界和工业界的热点研究方向。然而, 现有综述文献大多聚焦于单一飞行平台 (主要是多旋翼), 缺乏对多平台 (多旋翼、固定翼、VTOL) 的统一视角和系统性比较。此外, Sim-to-Real 迁移作为连接学术研究与实际应用的关键环节, 在现有综述中往往讨论不够深入。这些空白促使我们撰写本文, 旨在为研究人员和工程师提供一份全面、深入、实用的技术参考。

本文聚焦于强化学习在三种主要飞行平台 (多旋翼、固定翼和 VTOL) 控制中的应用, 系统综述该领域的研究现状、技术挑战和发展趋势。与现有的综述文献相比, 本文具有以下特点:

首先, **多平台视角**。现有综述大多聚焦于单一平台 (主要是多旋翼), 而本文同时涵盖三种主要飞行平台, 并分析了不同平台之间的技术迁移和借鉴关系。这种多平台视角有助于读者理解各类方法的通用性和特殊性, 促进跨平台的技术创新。

其次, **Sim-to-Real 专题**。本文将 Sim-to-Real 迁移作为独立章节进行深入讨论, 系统梳理了域随机化、域适应、系统辨识、元学习等核心技术。这是强化学习从学术研

究走向实际应用的关键环节，也是当前研究的热点和难点。

第三，**批判性分析**。本文不仅罗列已有研究成果，更注重对各类方法的优缺点、适用场景和局限性进行批判性分析。通过深入的方法对比和案例研究，帮助读者理解不同方法的选择依据和权衡考量。

第四，**实用指南**。本文提供了详细的算法超参数设置、仿真环境配置和奖励函数设计模板，为研究人员快速入门和工程实践提供直接可用的参考。

本文的主要贡献包括：

1. **系统梳理**：建立了覆盖多旋翼、固定翼、VTOL 三种平台的强化学习控制技术分类体系，涵盖从底层姿态控制到高层任务规划的完整技术栈；
2. **深入分析**：详细讨论了 Sim-to-Real 迁移的核心技术，总结了最佳实践和关键成功因素，为实际部署提供指导；
3. **挑战识别**：系统识别了当前面临的主要挑战 (样本效率、安全性、实时性、可解释性)，并展望了与大语言模型、神经辐射场等新兴技术的结合；
4. **实用资源**：提供了 RL 算法超参数设置、仿真环境配置、奖励函数设计模板等实用资源，降低了研究入门门槛。

1.4 本文组织结构

本文后续章节组织如下：

第 2 章介绍强化学习的基础理论，包括马尔可夫决策过程、值函数方法、策略梯度方法和 Actor-Critic 架构。这些理论基础为理解后续应用章节提供了必要的背景知识。

第 3 章深入讨论多旋翼平台的强化学习控制，涵盖姿态控制、位置跟踪、高速敏捷飞行、特殊任务 (悬挂载荷运输、编队控制) 等内容。多旋翼是目前 RL 飞行控制研究最成熟的平台。

第 4 章聚焦固定翼平台的强化学习控制，包括纵向控制 (高度、空速)、横侧向控制 (航向、协调转弯)、特殊机动 (失速改出、大攻角机动) 和编队飞行等任务。

第 5 章探讨 VTOL 平台的强化学习控制，重点讨论模态转换控制这一核心难题。VTOL 结合了多旋翼和固定翼的优点，但控制难度也相应增加。

第 6 章系统综述 Sim-to-Real 迁移技术，包括域随机化、域适应、系统辨识和在线适应等方法。这是 RL 飞行控制从仿真走向实际应用的关键环节。

第 7 章总结当前面临的主要挑战 (样本效率、安全性、实时性、可解释性)，展望新兴技术趋势 (大语言模型、神经辐射场、联邦学习、数字孪生)，并提出未来研究方向。

附录提供了实用的参考信息，包括 RL 算法超参数设置、仿真环境配置指南和奖励函数设计模板。

2 强化学习基础

强化学习是机器学习的一个重要分支，关注智能体如何在环境中通过试错学习最优行为策略。与监督学习不同，强化学习没有标注好的训练数据，而是通过与环境的交互获得奖励信号来指导学习过程。

2.1 马尔可夫决策过程

马尔可夫决策过程 (Markov Decision Process, MDP) 是强化学习的数学框架，为序贯决策问题提供了形式化描述。一个 MDP 由五元组 $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ 组成：

- $\mathcal{S}$: 状态空间，包含环境的所有可能状态；
- $\mathcal{A}$: 动作空间，包含智能体可以采取的所有动作；
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$: 状态转移概率；
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$: 奖励函数；
- $\gamma \in [0, 1]$: 折扣因子。

定义 2.1 (马尔可夫性质). 一个随机过程具有马尔可夫性质，如果未来状态的条件概率分布仅依赖于当前状态，而与过去状态无关：

$$P(S_{t+1}|S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = P(S_{t+1}|S_t, A_t) \quad (1)$$

强化学习的目标是找到一个策略 $\pi : \mathcal{S} \rightarrow \mathcal{A}$ ，使得累积折扣奖励最大化。累积折扣奖励定义为：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2)$$

为了评估策略的好坏，定义状态值函数 $V^\pi(s)$ 为从状态 s 开始遵循策略 π 的期望累积奖励：

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \quad (3)$$

类似地，定义动作值函数 $Q^\pi(s, a)$ 为在状态 s 采取动作 a 后遵循策略 π 的期望累积奖励：

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \quad (4)$$

定义 2.2 (贝尔曼方程 [85]). 值函数满足以下自洽方程，称为贝尔曼方程：

$$V^\pi(s) = \sum_a \pi(a|s) \left[\mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{P}(s'|s, a) V^\pi(s') \right] \quad (5)$$

$$Q^\pi(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{P}(s'|s, a) \sum_{a'} \pi(a'|s') Q^\pi(s', a') \quad (6)$$

最优值函数 $V^*(s)$ 和 $Q^*(s, a)$ 定义为所有策略中值函数的最大值：

$$V^*(s) = \max_{\pi} V^\pi(s), \quad Q^*(s, a) = \max_{\pi} Q^\pi(s, a) \quad (7)$$

2.2 值函数方法

值函数方法通过学习值函数来间接获得最优策略。这类方法的核心思想是：如果知道了最优动作值函数 $Q^*(s, a)$ ，最优策略就是选择使 Q^* 最大的动作。

2.2.1 Q 学习 [36]

Q 学习 (Q-Learning) 是一种经典的无模型强化学习算法，由 Watkins 于 1989 年提出，通过迭代更新来估计最优动作值函数。作为强化学习领域最具影响力的算法之一，Q 学习以其简洁的理论基础和良好的收敛性保证，成为值函数方法的代表性算法。

Q 学习的核心思想是通过时间差分 (Temporal Difference, TD) 学习来更新动作值函数的估计。其更新规则为：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (8)$$

其中 $\alpha \in (0, 1]$ 是学习率，控制新信息对旧估计的影响程度； $\gamma \in [0, 1]$ 是折扣因子，平衡即时奖励与未来奖励的重要性。

Q 学习的一个重要特性是其**离策略** (Off-Policy) 性质。这意味着 Q 学习可以在遵循任意探索策略 (如 ϵ -贪心策略) 的同时，学习最优策略的动作值函数。这一特性使得 Q 学习能够充分利用历史数据进行学习，而不仅限于当前策略产生的数据。

在飞行控制应用中，传统的表格型 Q 学习面临着维度灾难问题。飞行器的状态空间通常包含位置、速度、姿态、角速度等多个连续变量，离散化后会产生巨大的状态空间。为解决这一问题，研究人员提出了多种函数逼近方法，包括线性函数逼近和神经网络逼近。

收敛性分析：在满足一定条件下 (如所有状态-动作对被无限次访问、学习率满足 Robbins-Monro 条件)，Q 学习被证明能够收敛到最优动作值函数 Q^* 。然而，当结合函数逼近 (特别是非线性函数逼近如神经网络) 时，收敛性保证不再成立，这也是深度强化学习中理论研究的重要方向。

探索与利用的平衡：Q 学习中的探索策略对学习效果至关重要。常用的 ϵ -贪心策略以概率 ϵ 随机选择动作 (探索)，以概率 $1 - \epsilon$ 选择当前估计的最优动作 (利用)。在飞行控制中，过度的探索可能导致危险行为，因此需要谨慎设计探索策略，或采用安全强化学习方法。

近年来，研究人员提出了多种 Q 学习的改进算法，包括 Double Q-Learning (解决价值高估问题)、Prioritized Experience Replay (优先经验回放，提高样本效率)、Dueling DQN [14] (对偶网络架构，分别估计状态值和动作优势) 等。这些改进显著提升了 Q 学习在复杂任务中的性能，为其在飞行控制中的应用奠定了基础。

2.2.2 深度 Q 网络

深度 Q 网络 (Deep Q-Network, DQN) 由 DeepMind 于 2015 年提出，是深度学习与强化学习结合的开创性工作。DQN 首次成功将深度神经网络应用于高维感知输入 (如

原始像素) 的强化学习任务, 在 Atari 游戏上取得了超越人类专家的表现。这一突破为强化学习在复杂连续控制问题 (包括飞行控制) 中的应用开辟了新的道路。

DQN 的核心创新在于使用深度神经网络来近似动作值函数 $Q(s, a; \theta)$, 其中 θ 表示网络参数。为了解决深度神经网络与 Q 学习结合时的不稳定性问题, DQN 引入了以下关键技术:

经验回放 (Experience Replay): 将智能体的经验 $(s_t, a_t, r_{t+1}, s_{t+1})$ 存储在回放缓冲区 $\mathcal{D}$ 中, 训练时从中随机采样小批量样本。经验回放打破了数据之间的时序相关性, 提高了样本利用效率, 同时平滑了数据分布的变化, 有助于稳定训练过程。在飞行控制中, 经验回放使得智能体能够从过去的飞行经验中学习, 包括成功和失败的案例。

目标网络 (Target Network): 使用单独的网络 $Q(s', a'; \theta^-)$ 来计算目标值, 该网络的参数 θ^- 定期从主网络复制 (通常是每隔若干步), 而不是每一步都更新。这一机制减少了训练过程中的目标值变化, 提高了学习的稳定性。

DQN 的损失函数定义为均方误差:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (9)$$

Double DQN [37] 解决了 DQN 中 $\max$ 操作导致的价值高估问题。标准 DQN 使用同一个网络来选择和评估动作, 倾向于选择被高估价值的动作。Double DQN 使用主网络选择动作, 目标网络评估动作, 有效缓解了价值高估问题。其更新目标为:

$$Y_t^{DoubleDQN} = R_{t+1} + \gamma Q(S_{t+1}, \arg \max_a Q(S_{t+1}, a; \theta_t); \theta_t^-) \quad (10)$$

Dueling DQN [38] 采用对偶网络架构, 将动作值函数分解为状态值函数和动作优势函数: $Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$ 。这种分解使得网络能够更有效地学习哪些状态是有价值的, 而不必关心每个动作的具体影响。

在飞行控制应用中, DQN 及其变体主要用于离散动作空间的任務, 如航点选择、模式切换等高层决策。对于连续控制问题 (如姿态控制), 通常需要结合策略梯度方法或使用动作离散化技术。近年来, 研究人员也探索了将 DQN 与模型预测控制相结合的方法, 利用 DQN 进行长期规划, MPC 处理短期跟踪。

2.3 策略梯度方法

策略梯度方法直接参数化策略 $\pi_\theta(a|s)$, 并通过梯度上升来优化策略参数 θ 。与值函数方法相比, 策略梯度方法能够自然地处理连续动作空间。

2.3.1 策略梯度定理

策略梯度定理 (Policy Gradient Theorem) 是策略优化方法的理论基石, 由 Sutton 等人于 2000 年系统提出。该定理为直接优化参数化策略提供了梯度计算方法, 使得我们能够使用基于梯度的优化算法 (如随机梯度上升) 来改进策略。

策略梯度定理的核心结果提供了计算策略性能梯度的方法：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)] \quad (11)$$

其中 $\nabla_{\theta} \log \pi_{\theta}(a|s)$ 称为得分函数 (Score Function) 或似然比 (Likelihood Ratio), $Q^{\pi_{\theta}}(s, a)$ 是状态-动作值函数。该定理表明, 策略性能的梯度可以通过采样轨迹来估计, 而无需知道环境的转移模型。

REINFORCE [106] 算法是基于策略梯度定理的经典算法, 使用蒙特卡洛估计来计算梯度：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) G_t^{(i)} \quad (12)$$

其中 $G_t^{(i)}$ 是第 i 条轨迹从时刻 t 开始的累积回报。REINFORCE 算法是无偏的, 但方差较大, 因为使用了完整的轨迹回报作为权重。

基线减法 (Baseline Subtraction) 是降低方差的重要技术。通过引入与动作无关的基线 $b(s)$, 梯度估计变为：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) (Q^{\pi_{\theta}}(s, a) - b(s))] \quad (13)$$

常用的基线包括状态值函数 $V^{\pi_{\theta}}(s)$, 此时 $Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s) = A^{\pi_{\theta}}(s, a)$ 即为优势函数。使用优势函数作为权重可以显著降低梯度估计的方差, 提高学习效率。

在飞行控制应用中, 策略梯度方法特别适合处理连续动作空间问题。飞行器的控制输入 (如电机转速、舵面偏转角) 通常是连续变量, 策略梯度方法能够直接输出连续动作的概率分布参数 (如高斯分布的均值和方差), 避免了动作离散化带来的精度损失和维度灾难问题。

然而, 基本的策略梯度方法存在样本效率低的问题, 需要大量交互数据才能学习到有效策略。这一限制在真实飞行器上尤为突出, 因为实际飞行试验成本高、风险大。为解决这一问题, 研究人员发展了 Actor-Critic 方法, 结合值函数估计来降低方差, 提高样本效率。

2.3.2 近端策略优化 [15]

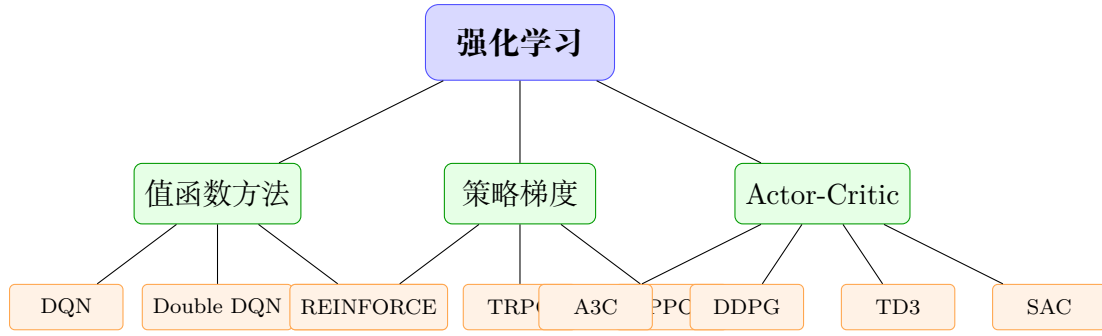
近端策略优化 (Proximal Policy Optimization, PPO [15]) 是 TRPO [35] 的简化实现, 在保持类似性能的同时大大降低了计算复杂度。PPO 使用裁剪目标函数来限制策略更新：

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (14)$$

其中 $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$ 是重要性采样比率, ϵ 是超参数 (通常取 0.1 或 0.2)。

PPO 因其简单性和有效性, 成为目前最常用的策略梯度算法之一, 在飞行控制领域得到了广泛应用。

图 2 展示了强化学习方法的分类框架。在飞行控制中, 由于连续动作空间和高维状态空间的特点, Actor-Critic 方法 (特别是 SAC [17] 和 PPO) 最为常用。



2.4 Actor-Critic 方法

Actor-Critic 方法结合了值函数方法和策略梯度方法的优点，使用两个网络：Actor(策略网络) 负责选择动作，Critic(值网络) 负责评估动作价值。

2.4.1 DDPG [16] 算法

深度确定性策略梯度 [16](Deep Deterministic Policy Gradient, DDPG) 由 Lillicrap 等人于 2016 年提出，是面向连续动作空间的 Actor-Critic 算法。DDPG 巧妙地将 DQN 的思想扩展到连续动作空间，通过使用确定性策略和双网络架构，实现了高效稳定的连续控制。

DDPG 的核心特点是使用确定性策略 $\mu_\theta(s)$ ，即对于给定状态，策略输出确定的动作而非动作分布。这一设计特别适合飞行控制等工程应用，因为确定性策略可以直接部署，无需采样。DDPG 包含四个网络：

- **Actor 网络** $\mu_\theta(s)$ ：输出确定性动作
- **Critic 网络** $Q_\phi(s, a)$ ：评估状态-动作对的价值
- **目标 Actor 网络** $\mu'_{\theta'}$ ：计算目标动作
- **目标 Critic 网络** $Q'_{\phi'}$ ：计算目标 Q 值

Actor 更新通过链式法则计算梯度：

$$\nabla_\theta J \approx \frac{1}{N} \sum_i \nabla_a Q(s_i, a|\phi)|_{a=\mu_\theta(s_i)} \nabla_\theta \mu_\theta(s_i) \quad (15)$$

Critic 更新采用与 DQN 类似的时间差分学习：

$$L(\phi) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\phi))^2 \quad (16)$$

其中目标值 $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1})|\phi')$ 。

探索策略：由于 DDPG 使用确定性策略，需要额外的探索机制。通常采用 Ornstein-Uhlenbeck 过程或简单的高斯噪声添加到时动作上： $a = \mu_\theta(s) + \mathcal{N}(0, \sigma^2)$ 。

在飞行控制应用中，DDPG 被广泛应用于多旋翼的姿态控制和轨迹跟踪任务。研究表明，DDPG 能够在仿真环境中学习到鲁棒的控制策略，但在 Sim-to-Real 迁移时面临挑战。

2.4.2 SAC 算法

软 Actor-Critic(Soft Actor-Critic, SAC) 由 Haarnoja 等人于 2018 年提出，是一种基于最大熵框架的 Actor-Critic 算法。SAC 在最大化累积奖励的同时最大化策略熵，鼓励探索并提高鲁棒性。SAC 因其出色的样本效率和稳定性，成为当前飞行控制领域最受欢迎的强化学习算法之一。

SAC 的核心思想是最大熵强化学习框架，目标函数为：

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_\pi [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))] \quad (17)$$

其中 $\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a|s)]$ 是策略熵， α 是温度参数控制探索程度。熵正则化鼓励策略保持随机性，避免过早收敛到次优解，同时提高了对模型不确定性的鲁棒性。

SAC 采用双 Q 网络技术来缓解 Q 值高估问题，使用两个 Critic 网络 Q_{ϕ_1} 和 Q_{ϕ_2} ，并取较小值作为目标：

$$y = r + \gamma \left(\min_{j=1,2} Q_{\phi_j}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right) \quad (18)$$

其中 $\tilde{a}'$ 是从策略网络中重参数化采样得到的动作。

重参数化技巧：SAC 使用重参数化技巧来估计策略梯度。动作被表示为确定性变换加上随机噪声： $a = f_\theta(s, \epsilon)$ ，其中 ϵ 是从固定分布（如标准正态分布）中采样的噪声。这使得梯度可以通过动作传播到策略参数。

自动温度调整：SAC 的原始实现需要手动调整温度参数 α 。后续研究提出了自动温度调整方法，通过求解以下约束优化问题来自动学习 α ：

$$\min_{\alpha} \mathbb{E}_{a \sim \pi_t^*} [-\alpha \log \pi_t^*(a|s) - \alpha \bar{\mathcal{H}}] \quad (19)$$

其中 $\bar{\mathcal{H}}$ 是目标熵。自动温度调整使得算法对不同任务更加鲁棒，减少了超参数调优的工作量。

在飞行控制应用中，SAC 展现出了优异的性能。与 DDPG 和 PPO 相比，SAC 通常具有更高的样本效率，能够在更少的交互数据下学习到高质量的控制策略。这一特性对于真实飞行器训练尤为重要，可以显著降低飞行试验成本。研究表明，SAC 在多旋翼姿态控制、固定翼着陆控制、VTOL 模态转换等任务中均取得了良好效果。

SAC 的熵最大化特性特别适合飞行控制中的安全探索。在训练初期，策略具有较高的随机性，可以充分探索状态空间；随着训练进行，策略逐渐收敛到确定性行为，但保

留了一定的随机性来应对不确定性。这种特性使得 SAC 训练的策略在面对风扰、模型误差等扰动时表现出更好的鲁棒性。

2.5 RL 控制框架

将强化学习应用于飞行控制需要精心设计状态观测、动作空间和奖励函数。

2.5.1 状态观测设计

状态观测的设计是强化学习飞行控制系统成功的关键因素之一。观测空间的选择需要在信息完整性和维度复杂度之间取得平衡。过于简单的观测可能无法提供足够的信息来学习有效策略，而过于复杂的观测则会增加学习难度，降低样本效率。

在飞行控制中，状态观测通常包括以下几个层次的信息：

运动学状态：这是最基本的观测信息，包括位置 $\mathbf{p} = [x, y, z]^T$ 、速度 $\mathbf{v} = [v_x, v_y, v_z]^T$ 、姿态（通常用四元数 $\mathbf{q} = [q_0, q_1, q_2, q_3]^T$ 或欧拉角 $[\phi, \theta, \psi]^T$ 表示）、角速度 $\boldsymbol{\omega} = [p, q, r]^T$ 。这些信息可以直接从惯性测量单元 (IMU) 和全球定位系统 (GPS) 获取。

目标信息：为了实现跟踪控制，观测中需要包含目标状态信息，如目标位置 $\mathbf{p}_{goal}$ 、目标速度 $\mathbf{v}_{goal}$ 、目标姿态等。通常使用相对量（如相对位置 $\mathbf{p}_{rel} = \mathbf{p}_{goal} - \mathbf{p}$ ）而非绝对量，这样可以提高策略的泛化能力。

环境信息：在某些任务中，环境信息对决策至关重要。例如，在风扰环境下飞行时，风速和风向的估计可以显著提高控制性能；在障碍物环境中，障碍物位置和距离信息是实现避障的基础。

历史信息：飞行器的动力学具有时序相关性，当前决策会影响未来状态。为了处理部分可观测问题，有时需要包含历史状态或历史动作作为观测。常用的方法包括：

- **状态堆叠：**将最近 k 个时刻的状态拼接成一个向量
- **递归网络：**使用 LSTM 或 GRU 等递归神经网络来处理时序信息
- **观测滤波：**使用卡尔曼滤波或粒子滤波来估计隐藏状态

观测归一化：不同观测量的量纲和数值范围差异很大（如位置可能以米为单位，姿态以弧度为单位），直接输入神经网络会导致学习困难。通常需要对观测进行归一化处理：

$$\tilde{o} = \frac{o - \mu_o}{\sigma_o} \quad (20)$$

其中 μ_o 和 σ_o 分别是观测的均值和标准差，可以通过在仿真中收集数据来估计。

在实际应用中，观测设计还需要考虑传感器噪声、延迟和故障等因素。鲁棒的观测设计应该能够在不完美感知条件下仍然支持有效的控制决策。

2.5.2 动作空间选择

动作空间的选择是强化学习飞行控制系统设计中的关键决策，直接影响学习难度、策略性能和系统安全性。动作空间的设计需要在控制精度和学习复杂度之间取得平衡，同时考虑实际执行器的物理约束。

根据控制任务的抽象层次，动作空间可以分为以下几类：

底层控制：直接输出执行器指令，如电机 PWM 信号、舵面偏转角度、推力矢量方向等。这种设计赋予强化学习最大的控制自由度，可以直接优化执行器输入，理论上能够达到最优性能。然而，底层控制也面临以下挑战：

- **高维度：**现代飞行器通常具有多个执行器 (如六旋翼有 6 个电机)，动作空间维度高，增加学习难度
- **物理约束：**执行器具有饱和限制 (最大/最小偏转角度) 和速率限制，需要在策略中考虑这些约束
- **安全性：**错误的执行器指令可能导致飞行器失控，需要安全探索机制

中层控制：输出期望姿态或期望角速度，由底层控制器 (如 PID) 跟踪。这种分层控制架构在实际工程中非常常见，具有以下优点：

- **降低维度：**动作空间维度显著降低 (如期望姿态只有 3 个自由度)
- **安全保证：**底层控制器提供基本的稳定性保证，强化学习专注于高层决策
- **模块化：**便于系统集成和调试，可以独立优化各层控制器

高层控制：输出高层指令，如目标航点、飞行模式切换、轨迹参数等。这种设计适用于任务规划层，强化学习负责决策而底层控制由传统方法处理。

混合动作空间：在某些复杂任务中，可能需要同时输出多个层次的控制指令。例如，同时输出姿态指令和推力指令，或者输出离散的模式选择和连续的控制指令。处理混合动作空间需要特殊的算法设计，如分层强化学习 [84] 或多智能体强化学习 [98]。

动作约束处理：实际飞行器的执行器具有物理限制，如舵面偏转角度限制、电机转速限制等。处理这些约束的方法包括：

- **动作裁剪：**将策略输出裁剪到合法范围内
- **惩罚函数：**对违反约束的动作给予惩罚
- **约束优化：**在策略优化中显式考虑约束条件
- **可执行动作空间：**使用 $\tanh$ 等激活函数将输出限制在 $[-1, 1]$ 范围内，然后映射到实际执行器范围

动作空间的选择应该基于具体任务需求和系统能力。对于需要精细控制的敏捷飞行任务，底层控制可能更合适；对于复杂的导航任务，中层或高层控制可以降低学习难度。

2.5.3 奖励函数设计

奖励函数是强化学习的核心，直接决定了学习到的策略行为。在飞行控制中，奖励函数的设计尤为关键，因为它需要在多个相互冲突的目标之间取得平衡，如跟踪精度、控制平滑性、能耗效率、安全性等。一个良好设计的奖励函数能够引导策略学习到既高效又安全的飞行行为。

飞行控制中常用的奖励设计原则包括：

跟踪误差惩罚：这是最基本的奖励组成部分，惩罚与目标状态的偏差。对于位置跟踪任务，可以使用：

$$r_{pos} = -\|\mathbf{p} - \mathbf{p}_{target}\|^2 \quad (21)$$

对于姿态跟踪任务：

$$r_{att} = -\|\mathbf{q} \ominus \mathbf{q}_{target}\|^2 \quad (22)$$

其中 $\ominus$ 表示四元数差运算。跟踪误差奖励通常使用平方误差来惩罚大偏差，但也可以使用线性或指数形式来调整对误差的敏感度。

控制平滑性：为了鼓励平滑控制，避免执行器频繁振荡，可以惩罚控制输入的剧烈变化：

$$r_{smooth} = -\lambda_1 \|\mathbf{u}\|^2 - \lambda_2 \|\dot{\mathbf{u}}\|^2 \quad (23)$$

其中 $\mathbf{u}$ 是控制输入， $\dot{\mathbf{u}}$ 是控制输入的变化率， λ_1 和 λ_2 是权重系数。平滑性奖励不仅提高了乘坐舒适性，还能减少执行器磨损和能耗。

能耗优化：对于电池供电的无人机，能耗是一个重要考虑因素。可以通过惩罚大控制输入来鼓励节能飞行：

$$r_{energy} = -\lambda_3 \|\mathbf{u}\|^2 \quad (24)$$

对于多旋翼，这直接对应于电机功率消耗；对于固定翼，则与舵面偏转和推力设置相关。

安全约束：飞行控制对安全性有极高要求，奖励函数必须包含对安全约束的考虑：

- **边界约束：**对超出安全飞行包线的行为给予大惩罚，如高度过低、速度过高/过低、攻角过大等
- **碰撞惩罚：**对接近障碍物或地面的行为给予惩罚
- **失速避免：**对接近失速条件的行为给予警告性惩罚
- **终端奖励：**在成功完成任务时给予正奖励，在失败（如坠毁）时给予大负奖励

稀疏奖励与密集奖励：奖励函数可以是稀疏的（只在任务完成或失败时给予奖励）或密集的（每个时间步都给予奖励）。稀疏奖励更符合某些任务的天然定义，但学习难度更大；密集奖励提供了更多的学习信号，但可能需要精心设计以避免不期望的行为。

课程学习 [86] 与奖励塑形：对于复杂任务，可以使用课程学习逐步增加任务难度，并相应调整奖励函数。奖励塑形 (Reward Shaping) 通过添加辅助奖励来引导学习过程，但需要注意避免改变最优策略。

多目标优化：实际飞行任务通常涉及多个目标，可以使用多目标强化学习方法，或设计加权和奖励函数：

$$r = w_1 r_{tracking} + w_2 r_{smooth} + w_3 r_{energy} + w_4 r_{safety} \quad (25)$$

权重 w_i 的选择反映了不同目标的相对重要性，需要根据具体应用进行调整。

表 2: 主流 RL 算法在飞行控制中的适用性对比

算法	连续动作	样本效率	稳定性	适用场景
DQN	否	中	中	离散决策
DDPG	是	中	低	连续控制
TD3 [18]	是	中	中	连续控制
SAC	是	高	高	连续控制
PPO	是	中	高	各种任务

表 2 对比了主流 RL 算法在飞行控制中的适用性。从表中可以看出，SAC 和 PPO 因其良好的稳定性和连续动作处理能力，在飞行控制中应用最为广泛。

3 多旋翼平台控制

多旋翼平台是强化学习在飞行控制中应用最广泛的领域。由于其结构简单、成本低廉、易于建模等特点，多旋翼成为验证新算法的理想平台。本节将深入讨论多旋翼平台的强化学习控制方法。

3.1 平台动力学特性

多旋翼飞行器的动力学模型是设计控制策略的基础。以四旋翼为例，其动力学可以用以下方程描述：

位置动力学：

$$m\ddot{\mathbf{p}} = -mg\mathbf{e}_3 + R(\mathbf{q})\mathbf{T}_b \quad (26)$$

其中 m 是飞行器质量， $\mathbf{p}$ 是位置， $R(\mathbf{q})$ 是从机体坐标系到惯性坐标系的旋转矩阵， $\mathbf{T}_b = [0, 0, T]^T$ 是机体坐标系下的推力， $T = \sum_{i=1}^4 k_i \omega_i^2$ 是总推力。

姿态动力学：

$$\mathbf{I}\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega} + \boldsymbol{\tau} \quad (27)$$

其中 $\mathbf{I}$ 是惯性矩阵， $\boldsymbol{\tau} = [\tau_x, \tau_y, \tau_z]^T$ 是力矩，由四个电机的差速产生。

四旋翼是**欠驱动系统**，即 4 个控制输入（4 个电机转速）需要控制 6 个自由度（3 个位置 + 3 个姿态）。这一特性使得控制设计具有挑战性，通常采用分层控制架构：内环控制姿态，外环控制位置。

3.2 姿态控制

姿态控制是多旋翼控制的基础，负责稳定飞行器的滚转、俯仰和偏航角。强化学习在姿态控制中的应用可以分为端到端方法和分层方法两类。

3.2.1 端到端 RL 姿态控制

端到端方法直接使用 RL 学习从状态到电机指令的映射，无需中间控制层。这种设计使得强化学习能够发现传统控制方法难以实现的非线性控制策略，充分利用神经网络的表达能力。

Koch 等人 (2019) 提出了基于深度 RL 的四旋翼姿态控制方法，使用 PPO 算法训练策略网络。该方法的状态空间包括当前姿态四元数 $\mathbf{q}$ 、角速度 $\boldsymbol{\omega}$ 和目标姿态 $\mathbf{q}_{target}$ ，动作空间是四个电机的归一化转速指令 $\mathbf{a} = [a_1, a_2, a_3, a_4]^T$ 。

奖励函数设计综合考虑了跟踪精度、控制平滑性和能耗：

$$r = -\|\mathbf{q}_{error}\|^2 - 0.01\|\boldsymbol{\omega}\|^2 - 0.001\|\mathbf{a}\|^2 \quad (28)$$

其中 $\mathbf{q}_{error}$ 是姿态误差四元数， $\boldsymbol{\omega}$ 是角速度， $\mathbf{a}$ 是动作。第一项惩罚姿态跟踪误差，第二项鼓励低角速度（减少振荡），第三项惩罚大控制输入（节能）。

实验结果表明，训练好的 RL 策略在跟踪性能上可以与精心调参的 PID 控制器相当，同时具有更好的抗干扰能力。在受到外部扰动（如阵风）时，RL 策略能够更快地恢复稳定状态。更重要的是，RL 策略可以方便地扩展到不同的任务目标，只需修改奖励函数即可，无需重新设计控制架构。

Sim-to-Real 迁移：端到端 RL 姿态控制面临的主要挑战是 Sim-to-Real 迁移。由于仿真模型与现实存在差异，直接在仿真中训练的策略可能在真实飞行器上表现不佳。解决这一问题的方法包括：

- **域随机化：**在训练时随机化仿真参数（如质量、惯性、电机响应等），使策略对模型不确定性具有鲁棒性
- **系统辨识：**通过实验数据辨识真实飞行器的动力学模型，提高仿真保真度
- **在线适应：**部署后在真实环境中进行少量在线学习，适应真实动力学

安全性考虑：端到端控制直接输出电机指令，错误的指令可能导致飞行器失控。因此，在实际部署时需要考虑安全机制，如动作裁剪、紧急降落模式、人工接管等。

3.2.2 与传统 PID 对比

与传统 PID 控制器相比，RL 姿态控制在多个方面展现出独特优势，同时也存在一些局限性。深入理解这些差异有助于在实际应用中选择合适的控制方法。

优势：

1. **非线性处理能力**：PID 控制器本质上是线性控制器，在处理高度非线性的飞行器动力学时性能受限。RL 使用神经网络作为函数逼近器，能够自然地学习非线性控制律，在整个飞行包线内保持较好性能。
2. **自适应能力**：传统 PID 需要针对不同飞行条件 (如不同载荷、不同高度) 分别调参。RL 策略可以通过在多样化条件下训练，学习到自适应的控制行为，自动适应工况变化。
3. **多目标优化**：PID 通常只能优化单一目标 (如跟踪误差)。RL 的奖励函数可以灵活地编码多个目标 (跟踪精度、能耗、平滑性等)，学习多目标权衡的最优策略。
4. **端到端优化**：RL 直接优化从传感器输入到执行器输出的映射，避免了分层控制中各层目标不一致的问题。

局限性：

1. **可解释性差**：神经网络控制器的决策过程难以解释，这与航空领域严格的认证要求相冲突。
2. **训练成本高**：需要大量计算资源进行训练，且训练过程存在不确定性。
3. **安全性挑战**：训练过程中的探索可能导致危险行为，需要专门的安全机制。
4. **泛化能力限制**：训练策略可能过拟合到训练分布，对超出分布的情况处理能力有限。

在实际应用中，混合方法 (如 RL 与传统控制结合) 可能是更实用的选择。例如，使用 RL 进行高层决策，底层仍使用 PID 保证基本稳定性；或者使用 RL 学习 PID 参数的在线调整策略。

自适应能力：RL 策略可以根据当前状态自适应地调整控制增益，而 PID 使用固定增益。

非线性补偿：神经网络可以学习系统的非线性特性，而 PID 基于线性近似。

多目标优化：RL 的奖励函数可以灵活地编码多个目标，如跟踪精度、能耗、平滑性等。

然而，RL 方法也存在一些局限性：需要大量训练数据、难以提供稳定性保证、对奖励函数设计敏感等。因此，实际应用中常采用 RL 与传统控制器结合的方式。

3.3 位置与轨迹跟踪

位置控制负责将飞行器引导到目标位置或跟踪期望轨迹。由于四旋翼的欠驱动特性，位置控制通常通过生成期望姿态来实现。

3.3.1 分层控制架构

分层控制是工业界最常用的架构，将位置控制和姿态控制解耦为两个相对独立的控制环。这种架构的优势在于降低了控制设计的复杂度，便于模块化开发和调试，同时也为强化学习的应用提供了自然的分层接口。

位置控制器：根据位置误差生成期望推力方向和大小，进而计算出期望滚转角和俯仰角。在 RL 框架下，位置策略可以学习非线性的位置-姿态映射关系，考虑速度、加速度等动态约束。状态空间包括当前位置、速度、目标位置、目标速度等，动作空间是期望姿态（滚转角、俯仰角）和期望推力。奖励函数通常设计为：

$$r_{pos} = -\|\mathbf{p} - \mathbf{p}_{target}\|^2 - \lambda_1 \|\mathbf{v} - \mathbf{v}_{target}\|^2 - \lambda_2 \|\mathbf{a}_{cmd}\|^2 \quad (29)$$

其中 $\mathbf{a}_{cmd}$ 是加速度指令，用于惩罚过于激进的控制。

姿态控制器：跟踪期望姿态，生成电机指令。姿态控制环通常运行在更高的频率（如 500Hz-1kHz），负责快速稳定飞行器姿态。RL 姿态策略可以学习考虑执行器饱和、速率限制等约束的最优控制律。

分层训练策略：在 RL 框架下，可以分别训练位置策略和姿态策略，或者训练一个端到端策略直接输出电机指令。研究表明，分层训练通常更稳定，因为每个子任务的复杂度较低，且可以利用传统控制器作为基准进行初始化。而端到端训练可能获得更优的性能，因为避免了分层控制中各层目标不一致的问题。

时间尺度分离：分层控制的有效性依赖于位置控制和姿态控制的时间尺度分离。姿态动态通常比位置动态快一个数量级，这使得内环姿态控制可以近似为瞬时响应，简化了外环位置控制的设计。在 RL 训练中，可以通过不同的控制周期和奖励折扣因子来实现这种时间尺度分离。

混合架构：实际应用中常采用混合架构，即底层使用传统 PID 或 LQR 保证基本稳定性，高层使用 RL 进行轨迹优化和任务决策。这种架构结合了传统方法的安全性和 RL 的优化能力，是目前最实用的方案之一。

3.3.2 高速敏捷飞行

高速敏捷飞行是无人机控制中最具挑战性的任务之一，要求飞行器在复杂环境中以高速进行急转弯、穿越狭小间隙等机动动作。Loquercio 等人 (2021) 在 Nature 上发表的开创性工作展示了深度 RL 在这一领域的巨大潜力。他们训练了一个视觉-运动策略，仅使用机载摄像头和 IMU 数据，就能在复杂环境中以超过 40km/h 的速度飞行，性能达到人类世界冠军级水平。

任务定义与挑战：高速敏捷飞行的核心挑战在于反应时间极短。以 40km/h 的速度飞行时，飞行器每秒移动超过 11 米，而典型的人类反应时间为 200-250 毫秒，这意味着在反应期间飞行器已经移动了 2-3 米。因此，高速飞行要求控制策略具有极低的延迟和极高的预测能力。

方法核心创新：

1. **仿真到现实迁移**：使用大规模的域随机化，在仿真中训练能够直接迁移到真实环境的策略。随机化参数包括光照条件、纹理、相机参数、动力学参数等，使策略对各种环境变化具有鲁棒性。
2. **感知-动作联合学习**：策略网络同时处理视觉输入和运动控制，实现了端到端的感知-动作映射。网络架构包括一个卷积神经网络 [95] 处理图像，一个全连接网络处理 IMU 数据，融合后输出控制指令。
3. **模拟人类飞行员**：通过模仿学习 [101] 初始化策略，然后使用 RL 进行微调，使策略具有类似人类的飞行风格。人类飞行员的数据不仅提供了良好的初始化，还引入了人类对环境的理解。

网络架构：策略网络采用非对称 Actor-Critic 架构。Actor 网络处理视觉输入和 IMU 数据，输出控制指令；Critic 网络在训练时可以访问额外的状态信息 (如真实位置、速度)，提供更准确的值估计，加速学习。

奖励设计：奖励函数综合考虑了多个目标：

$$r = r_{progress} + r_{collision} + r_{smooth} + r_{speed} \quad (30)$$

其中 $r_{progress}$ 奖励沿路径前进， $r_{collision}$ 惩罚碰撞， r_{smooth} 鼓励平滑控制， r_{speed} 奖励高速飞行。

实验结果：在真实世界的竞赛中，该方法的飞行性能可以与人类世界冠军级飞行员相媲美，在某些任务上甚至超越人类。更重要的是，该方法展示了从零开始、仅通过仿真训练就能达到专家级性能的可能性。

3.4 特殊任务控制

除了基本的姿态和位置控制，强化学习还被应用于各种特殊任务。

3.4.1 着陆控制与地面效应处理

着陆是无人机飞行中最危险的阶段之一，据统计，超过 50

地面效应的物理机制：当多旋翼接近地面时，旋翼产生的下洗气流受到地面阻挡，形成高压气垫。这种效应使得在相同电机转速下产生更大的推力。地面效应的强度与飞行高度、旋翼直径、地面类型 (硬地面、草地、水面等) 有关。精确建模地面效应需要考虑复杂的流体力学，计算成本高昂。

Neural Lander 方法：该方法使用神经网络学习地面效应模型，并结合 RL 进行控制策略训练。具体来说，首先通过仿真或实验数据训练一个神经网络来预测地面效应对推力的影响；然后将这个神经网络集成到仿真环境中，使用 RL 训练着陆控制策略。训练好的策略能够精确预测地面效应对推力的影响，并相应调整控制指令，实现了精确稳定的着陆控制。

深度强化学习着陆：除了建模地面效应，深度 RL 还可以直接从数据中学习地面效应下的控制策略。通过在仿真中模拟不同高度、不同地面条件下的飞行，策略可以隐式地学习地面效应的影响，无需显式建模。研究表明，这种端到端学习方法在处理复杂地面效应时比基于模型的方法 [91] 更加鲁棒。

传感器融合：精确着陆还需要融合多种传感器信息，包括高度计 (超声波、激光雷达、气压计)、视觉 (检测地面纹理变化)、IMU 等。深度 RL 可以学习最优的传感器融合策略，根据当前状态动态调整对不同传感器的信任度。

3.4.2 悬挂载荷运输

悬挂载荷运输是无人机在物流领域的重要应用，具有广阔的商业前景。与单独飞行相比，悬挂载荷后的系统动力学发生显著变化：载荷的摆动与飞行器运动耦合，形成了复杂的非线性动力学系统。这种耦合效应使得控制难度大幅增加，传统的基于模型的控制方法难以适应载荷质量、绳索长度等参数的变化。

耦合动力学：悬挂载荷系统的动力学可以用双体系统描述。飞行器的运动通过绳索传递给载荷，引起载荷的摆动；同时，载荷的摆动反作用于飞行器，影响其姿态和位置。这种双向耦合使得系统具有非最小相位特性，控制输入 (飞行器加速度) 首先产生与期望方向相反的反应 (载荷向后摆动)，然后才产生期望反应。

摆动抑制策略：RL 方法可以学习考虑载荷摆动的控制策略，在运输过程中抑制摆动，实现平稳的货物运送。奖励函数通常包括飞行器位置跟踪误差、载荷摆动角度、控制能耗等项。通过在奖励中显式惩罚载荷摆动，策略可以学会平滑的飞行轨迹，避免急加速和急转弯。

模型无关优势：与传统基于模型的方法相比，RL 不需要精确的载荷动力学模型。载荷质量、绳索长度、空气阻力等参数在实际应用中往往不确定或时变，RL 策略可以通过在多样化条件下训练，学习到对这些变化具有鲁棒性的控制行为。

多机协同搬运：对于重型载荷，可以使用多架无人机协同搬运。这引入了额外的协调复杂性，需要解决控制分配、力协调、容错等问题。多智能体强化学习 (MARL) 为这类问题提供了框架，可以学习分布式控制策略，使每架无人机仅根据局部信息做出决策。

3.4.3 故障容错控制

多旋翼飞行器具有执行器冗余特性，这为其提供了天然的故障容错能力。以四旋翼为例，虽然正常情况下使用四个电机控制六个自由度 (欠驱动)，但在一个电机完全失效后，系统仍然保持一定的可控性，可以实现紧急降落或返航。然而，故障后的动力学发生显著变化，控制分配矩阵失效，传统线性控制器难以适应这种非线性、非对称的动力学特性。

故障模式分析：多旋翼常见的故障模式包括：单电机失效、电机部分失效 (推力下降)、旋翼损坏 (效率降低)、传感器故障等。不同故障模式对飞行性能的影响不同，需要

针对性的控制策略。

RL 故障容错方法：RL 方法可以学习故障情况下的控制策略。通过在仿真中模拟各种故障场景进行训练，策略能够学会利用剩余执行器维持稳定飞行。训练时，在随机时刻注入随机类型的故障，迫使策略学习故障检测和自适应控制 [83]。

控制重分配：故障后需要重新分配控制权限。例如，四旋翼单电机失效后，对角电机需要产生更多推力来补偿偏航力矩，同时通过姿态倾斜来产生水平方向的力。RL 策略可以隐式地学习这种控制重分配，无需显式求解复杂的控制分配优化问题。

故障检测与诊断：及时的故障检测对于容错控制至关重要。RL 可以结合故障检测算法，当检测到故障时切换到相应的容错控制模式。研究表明，训练好的策略在单电机失效情况下仍能保持悬停和缓慢移动，为安全降落争取时间。

混合方法：实际应用中，常采用混合方法：正常状态下使用传统控制器，当检测到故障时切换到 RL 容错控制器。这种架构结合了传统控制器的效率和 RL 的适应性。

3.5 多机协同控制

多无人机协同控制是 RL 应用的另一个重要方向，涉及编队控制、协同搬运、协同探索等任务。

3.5.1 编队控制

编队控制要求多架无人机保持特定的几何构型，是无人机集群应用的基础技术。编队控制在军事侦察、环境监测、搜救任务等领域具有重要应用价值。与单机控制相比，编队控制引入了额外的协调复杂性，需要考虑无人机之间的通信、避碰、角色分配等问题。

RL 方法可以学习分布式控制策略，每架无人机仅根据局部信息（自身状态和邻居状态）做出决策，无需中央控制器。这种分布式架构具有良好的可扩展性和容错性。

多智能体强化学习 (Multi-Agent Reinforcement Learning, MARL) 框架适用于此类问题。常用的方法包括：

独立学习 (Independent Learning)：每架无人机独立训练自己的策略，将其他无人机视为环境的一部分。这种方法实现简单，但忽略了多智能体之间的交互，可能导致非平稳性问题（其他智能体的策略变化导致环境动态变化）。

参数共享 (Parameter Sharing)：所有无人机共享同一个策略网络参数，减少训练复杂度，同时保证各无人机的策略一致性。参数共享显著减少了需要训练的参数数量，提高了样本效率。

集中训练分散执行 (Centralized Training with Decentralized Execution, CTDE)：训练时使用全局信息（所有无人机的状态），执行时仅使用局部信息。这种方法在训练时可以利用全局信息加速学习，执行时保持分布式特性。MADDPG (Multi-Agent DDPG) 是这类方法的典型代表。

通信学习：在编队控制中，无人机之间需要交换信息以协调动作。RL 可以学习何时通信、通信什么内容，以及如何利用接收到的信息做出决策。

3.5.2 协同搬运

多无人机协同搬运重物是展示多机协同能力的典型任务，也是无人机物流应用的潜在场景。与单机搬运相比，协同搬运可以处理更重的载荷，同时提供冗余性 (部分无人机失效后仍可完成任务)。然而，协同搬运引入了精确的力协调问题，每架无人机的推力需要与其他无人机协调，以保持载荷稳定并跟踪期望轨迹。

力协调挑战：协同搬运的核心挑战是力协调。载荷的运动由所有无人机的合力决定，任何不协调的推力都会导致载荷倾斜或摆动。此外，无人机之间的气动干扰 (如下洗气流) 也会影响协调控制。

集中式控制：在集中式控制架构中，中央控制器收集所有无人机的状态信息，计算最优控制指令并分发给各无人机。这种方法可以实现全局最优，但对通信要求高，且存在单点故障风险。

分布式 RL：RL 方法可以学习分布式协同控制策略，每架无人机仅根据局部信息 (自身状态、载荷状态、邻居状态) 做出决策。通过适当的奖励设计 (如惩罚载荷倾斜、奖励位置跟踪)，策略可以学会隐式的力协调，无需显式的控制分配优化。

实验验证：实验表明，基于 RL 的协同控制方法能够处理复杂的耦合动力学，实现稳定的协同搬运。在仿真中，训练好的策略可以适应载荷质量变化、无人机数量变化等情况。

3.6 方法对比与局限性

表 3: 多旋翼 RL 控制方法性能对比			
方法	跟踪精度	鲁棒性	计算成本
PID	中	低	极低
LQR	中	中	低
MPC	高	高	高
DDPG	中	中	中
SAC	高	高	中
PPO	高	高	中

表 3对比了不同控制方法在多旋翼平台上的性能。从表中可以看出，RL 方法 (特别是 SAC 和 PPO) 在跟踪精度和鲁棒性方面表现优异，同时计算成本适中，适合实际部署。

然而，当前方法仍存在以下局限性：

样本效率：训练有效的策略需要大量的仿真数据，对计算资源要求较高。

安全性保证：缺乏对 RL 策略稳定性的形式化证明，难以满足航空认证要求。

泛化能力：训练好的策略在面对与训练分布差异较大的情况时性能下降。

可解释性：神经网络控制器的决策过程难以解释，不利于调试和维护。

4 固定翼平台控制

固定翼飞行器具有与多旋翼截然不同的动力学特性。其前飞效率高、航程远，适合长航时任务。然而，固定翼控制面临飞行包线约束、失速风险等独特挑战。

4.1 平台动力学特性

固定翼飞行器的动力学可以用六自由度刚体方程描述，气动力和力矩是速度、攻角、侧滑角等的复杂函数。

纵向动力学：

$$m\dot{V} = T \cos \alpha - D - mg \sin \gamma \quad (31)$$

$$mV\dot{\gamma} = T \sin \alpha + L - mg \cos \gamma \quad (32)$$

$$\dot{q} = \frac{M}{I_{yy}} \quad (33)$$

其中 V 是空速， γ 是航迹角， α 是攻角， L 和 D 是升力和阻力， M 是俯仰力矩， T 是推力。

固定翼控制的关键约束包括：

失速约束：攻角必须小于失速攻角，否则升力急剧下降。

速度约束：飞行速度必须在最小速度和最大速度之间。

过载约束：法向过载必须在结构强度允许范围内。

4.2 纵向控制

纵向控制包括高度控制、空速控制和垂直速率控制。

4.2.1 高度与空速控制

高度和空速控制是固定翼最基本的控制任务，也是飞行控制的核心功能之一。传统方法使用总能量控制 (Total Energy Control System, TECS)，其基本思想是将推力用于控制总能量 (高度 + 速度之和)，将俯仰角用于控制能量分配 (高度与速度之间的转换)。这种解耦设计简化了控制设计，但在复杂工况下性能受限。

RL 方法可以学习更优的能量管理策略，通过神经网络学习非线性的能量管理律。研究表明，基于 RL 的控制器具有以下优势：

- **非线性补偿**：能够学习气动特性的非线性变化，在不同速度、高度条件下保持最优性能
- **多目标优化**：在奖励函数中同时编码跟踪精度、燃油消耗、乘客舒适性等多个目标
- **自适应能力**：能够适应飞行器质量变化 (如燃油消耗)、大气条件变化等

通过在奖励函数中显式编码燃油消耗项：

$$r_{fuel} = -\lambda_{fuel} \cdot T \quad (34)$$

其中 T 是推力，策略可以学会更节能的飞行方式。研究表明，基于 RL 的控制器在相同任务下可以比传统 TECS 方法节省 5-10% 的燃油，这在长航时任务中具有显著的经济效益。

风扰适应：固定翼对风扰较为敏感，特别是在低空飞行时。RL 策略可以通过在仿真中训练时加入风扰模型，学习到对风扰具有鲁棒性的控制策略。Neural-Fly [25] 等方法进一步结合在线适应，能够实时估计风场参数并调整控制策略。

4.2.2 着陆控制

着陆是固定翼飞行中最复杂的阶段，也是事故率最高的飞行阶段。着陆过程涉及精确的高度、速度、姿态和下沉率控制，需要飞行员或自动驾驶系统具备高超的技能。

传统自动着陆系统通常采用分段控制策略：

1. **进近阶段**：保持恒定下滑角 (通常为 3 度) 和恒定速度
2. **flare 阶段**：在接近地面时拉起机头，减小下沉率
3. **滑跑阶段**：接地后使用刹车和反推减速

flare 机动是着陆的关键阶段，需要精确控制下沉率从约 3-4 英尺/秒减小到接近零。过早或过晚拉起都会导致重着陆或平飘过长。RL 方法可以学习 flare 机动策略，根据当前高度、下沉率、空速等状态，实时调整俯仰角指令，实现平稳接地。

深度强化学习着陆：Kaufmann 等人 (2020) 提出了基于深度 RL 的固定翼着陆控制方法。该方法使用 SAC 算法训练策略网络，状态包括高度、空速、下沉率、到跑道距离等，动作是升降舵和油门指令。奖励函数设计为：

$$r = r_{touchdown} + r_{sink_rate} + r_{position} + r_{attitude} \quad (35)$$

其中各项分别奖励正确的接地时机、适当的下沉率、跑道中心线对齐和合适的姿态。

实验结果表明，训练好的 RL 策略能够在各种初始条件下实现稳定着陆，性能接近人类专家飞行员。更重要的是，RL 策略能够处理传统方法难以应对的异常情况，如强侧风、风切变等。

地面效应处理：在接近地面时 (通常小于一个翼展高度)，地面效应会改变机翼的气动特性，增加升力、减小诱导阻力。RL 策略可以通过学习地面效应模型，在 flare 阶段自动补偿这种影响，提高着陆精度。

4.3 横侧向控制

横侧向控制包括航向控制、协调转弯和航迹跟踪。

4.3.1 航向与协调转弯

协调转弯 (Coordinated Turn) 是固定翼飞行中的基本机动，要求飞行器在转弯时保持侧滑角为零，即乘客感受到的力垂直于机舱地板 (没有侧向滑动感)。协调转弯需要同时控制副翼 (产生滚转力矩) 和方向舵 (产生偏航力矩)，实现无侧滑转弯。

物理原理：当飞行器倾斜滚转时，升力的水平分量提供向心力使飞行器转弯。如果不协调副翼和方向舵，飞行器会出现侧滑：内侧滑 (向转弯中心滑动) 或外侧滑 (远离转弯中心滑动)。侧滑会导致阻力增加 (由方向舵修正产生)、乘客不适、甚至失速风险。

传统协调控制：传统方法使用转弯协调仪 (Turn Coordinator) 测量侧滑角或滚转率，通过偏航阻尼器和方向舵自动控制实现协调转弯。然而，传统控制器的参数需要针对不同飞行条件调整，且难以处理大角度机动。

RL 协调转弯：RL 方法可以学习协调转弯策略，同时控制副翼和方向舵，实现无侧滑转弯。状态空间包括滚转角、偏航角、侧滑角、滚转率、偏航率等，动作空间是副翼和方向舵指令。奖励函数设计为：

$$r = -\lambda_1 \|\beta\|^2 - \lambda_2 \|\dot{\psi} - \dot{\psi}_{cmd}\|^2 - \lambda_3 \|\phi - \phi_{cmd}\|^2 \quad (36)$$

其中 β 是侧滑角， $\dot{\psi}$ 是偏航率， ϕ 是滚转角。第一项惩罚侧滑 (协调转弯目标)，第二项跟踪期望偏航率 (转弯速率目标)，第三项跟踪期望滚转角 (倾斜角目标)。

大角度机动：对于大角度协调转弯 (如 45 度倾斜角)，气动特性进入非线性区域，传统线性控制器性能下降。RL 策略可以通过在训练中包含大角度机动，学习非线性的协调控制律。

4.3.2 航迹跟踪

航迹跟踪 (Path Following) 要求飞行器跟踪预设的航路点序列，是自主飞行的核心功能。与轨迹跟踪 (Trajectory Tracking) 不同，航迹跟踪只要求飞行器通过指定路径点，而不严格要求通过的时间和速度。

跟踪误差定义：航迹跟踪的关键是定义跟踪误差。常用的方法包括：

- **横向误差：**飞行器到期望路径的垂直距离
- **纵向误差：**沿路径方向的进度误差
- **航向误差：**当前航向与期望路径方向的夹角

非线性制导律：传统航迹跟踪使用非线性制导律 (Nonlinear Guidance Law, NLG)，根据横向误差和航向误差计算横向加速度指令。NLG 具有良好的收敛性和鲁棒性，但需要精确的参数调优。

RL 航迹跟踪：RL 方法可以学习路径跟踪策略，根据当前位置、航向和路径几何计算控制指令（通常是滚转角指令或副翼指令）。深度神经网络可以学习复杂的非线性制导律，自动适应不同的路径形状（直线、圆弧、过渡曲线）。

三维航迹跟踪：对于需要考虑高度的三维航迹跟踪，RL 可以同时学习横向和纵向控制策略，或者使用分层架构分别控制。奖励函数需要综合考虑横向跟踪误差、高度跟踪误差、航向对齐、控制平滑性等。

4.4 特殊机动控制

4.4.1 失速改出

失速 (Stall) 是固定翼飞行中的危险状态，当攻角超过临界值时，机翼上表面气流分离，升力急剧下降、阻力大幅增加，导致飞行器失去支撑而坠落。失速改出是飞行员必须掌握的基本技能，也是自动驾驶系统的重要功能。

失速征兆：现代飞行器配备有失速警告系统，通过测量攻角、空速、抖杆等参数检测即将发生的失速。失速征兆包括：空速过低、攻角过大、操纵面效率下降、机身抖动等。

传统改出程序：标准失速改出程序包括：(1) 推杆减小攻角至安全范围；(2) 加满油门增加空速；(3) 改平坡度；(4) 当空速恢复后柔和拉杆改出俯冲。这个程序简单有效，但在低空（如起飞或着陆阶段）可能高度不足，需要更激进的改出动作。

RL 失速改出：RL 方法可以学习失速检测和改出策略，在检测到失速征兆时自动执行改出机动。与传统固定程序不同，RL 策略可以根据当前状态（高度、空速、攻角、姿态）动态调整改出动作，在低空情况下采取更激进的推杆动作以快速恢复升力，在高空情况下采取更保守的动作以避免过度俯冲。

深度学习方法：深度神经网络可以学习从传感器数据到改出动作的端到端映射，无需显式的失速检测。研究表明，基于深度 RL 的失速改出策略在各种初始条件下都能成功改出，性能优于固定程序。

预防性控制：除了改出，更重要的是预防失速。RL 策略可以学习在接近失速边界时自动调整飞行状态，如减小爬升角、增加空速等，避免进入失速状态。

4.4.2 大攻角机动

大攻角机动 (High Angle of Attack Maneuver) 超出常规飞行包线，进入气动非线性区域。这类机动在特技飞行、空战机动、规避障碍等场景中有应用，但控制难度大、风险高。

气动非线性：在大攻角条件下，机翼和机身产生复杂的分离流、涡流，气动特性高度非线性且难以建模。升力系数随攻角的变化不再是线性的，可能出现升力下降（失速前）或波动（涡破裂）。

耦合效应：大攻角机动通常涉及多个轴的耦合运动，如快速拉杆同时伴随滚转和偏航。这种耦合效应使得控制设计复杂，传统解耦控制方法失效。

推力矢量：现代战斗机配备推力矢量喷口，可以在大攻角时提供额外的控制力矩，实现超机动（如眼镜蛇机动、落叶飘）。推力矢量的控制增加了控制维度和复杂性。

RL 大攻角控制：RL 方法可以学习大攻角机动策略，用于特技飞行或规避机动。通过在仿真中训练，策略可以学习复杂的气动特性，无需精确的解析模型。状态空间包括攻角、侧滑角、滚转角、角速度、空速等，动作空间包括舵面偏转和推力矢量角度。

安全训练：大攻角机动训练风险高，通常在仿真中进行。域随机化可以帮助策略学习对各种气动不确定性的鲁棒性。在线适应可以进一步调整策略以适应真实飞行器的气动特性。

4.5 方法对比与局限性

固定翼 RL 控制与多旋翼相比有以下特点：

动力学复杂性：固定翼具有更复杂的气动特性，需要更精确的模型。

控制冗余度低：固定翼控制面失效后容错能力较弱。

飞行包线约束：必须严格遵守失速速度、最大过载等约束。

长时域决策：轨迹优化涉及长时域的决策问题。

5 VTOL 平台控制

垂直起降 (VTOL) 飞行器结合了多旋翼的垂直起降能力和固定翼的高效前飞性能。然而，这种混合特性也带来了独特的控制挑战，特别是模态转换控制。

5.1 平台动力学特性

VTOL 飞行器有多种构型，包括倾转旋翼、倾转机翼和尾座式。不同构型的动力学特性差异很大，但共同特点是需要在悬停模式和前飞模式之间进行转换。

模态转换过程中，飞行器的气动特性发生剧烈变化。在悬停模式，飞行器类似于多旋翼；在前飞模式，飞行器类似于固定翼；在过渡模式，气动特性复杂且时变。

5.2 模态转换控制

模态转换控制是 VTOL 控制的核心难题，需要在保证稳定性的前提下实现平滑过渡。

5.2.1 转换走廊与轨迹规划

转换走廊 (Transition Corridor) 是 VTOL 飞行器安全进行模态转换的速度-高度包线范围。在这个包线内, 飞行器具有足够的操纵余度来完成转换并保持稳定。转换走廊的边界由多个因素决定: 最小转换速度 (低于此速度机翼无法提供足够升力)、最大转换速度 (结构强度限制)、最小高度 (应急着陆要求) 等。转换走廊的概念对于确保转换过程的安全性至关重要, 飞行员或自动驾驶系统必须严格遵守这些约束。

转换走廊的确定: 转换走廊的边界通常通过飞行试验和仿真分析确定。下边界由最小转换速度决定, 低于此速度机翼产生的升力不足以支撑飞行器重量; 上边界由结构强度限制, 高速转换可能导致过大的气动载荷; 侧边界由操纵余度决定, 必须保证在转换过程中有足够的控制能力应对扰动。

RL 方法可以学习最优转换轨迹规划策略。与传统基于模型的轨迹优化方法相比, RL 方法具有以下优势:

- **计算效率:** 训练好的策略网络前向推理速度快, 适合实时应用。传统轨迹优化需要在线求解优化问题, 计算成本高; 而 RL 策略只需要一次前向传播即可得到控制指令。
- **鲁棒性:** 通过在训练中引入不确定性, 策略可以学习到对模型误差具有鲁棒性的轨迹。这对于 VTOL 特别重要, 因为转换过程中的气动特性难以精确建模。
- **多目标优化:** 可以同时优化转换时间、能耗、乘客舒适性等多个目标。RL 的奖励函数可以灵活地编码这些目标, 学习多目标权衡的最优策略。

分层 RL 方法: VTOL 转换控制通常采用分层架构。高层策略负责决策 (何时开始转换、目标速度是多少), 低层策略负责跟踪 (如何控制执行器实现期望状态)。这种分层设计降低了学习难度, 同时提供了一定的安全保证。高层策略的运行频率较低 (如 10Hz), 负责规划; 低层策略的运行频率较高 (如 100Hz), 负责执行。

5.2.2 离线 RL 方法

离线强化学习 [94](Offline RL), 也称为批量强化学习 (Batch RL) 或从演示学习 (Learning from Demonstration), 是一种从固定数据集中学习策略的方法, 无需在训练过程中与环境交互。这对于 VTOL 模态转换这种高风险场景特别有价值, 因为可以避免在真实系统上进行危险的探索性飞行, 显著降低开发成本和风险。

离线 RL 的优势:

- **安全性:** 无需在真实系统上进行探索, 避免了危险动作导致的事故风险
- **数据利用:** 可以充分利用历史数据, 包括人类飞行员演示、传统控制器记录、之前的飞行日志等

- **可重复性**：训练过程确定且可重复，便于调试和优化

离线 RL 面临的主要挑战是**分布偏移** (Distribution Shift) 问题：学习到的策略可能会采取与数据收集策略不同的动作，从而遇到与训练数据分布不同的状态，导致性能下降或不稳定行为。这被称为“外推误差”(Extrapolation Error)。

解决分布偏移问题的方法包括：

- **保守估计**：在值函数估计时引入保守性，避免对分布外状态的高估。CQL [94](Conservative Q-Learning) 通过惩罚对分布外动作的 Q 值估计来实现保守性。
- **行为克隆 [101] 正则化**：限制学习策略与数据收集策略的差异，防止策略偏离数据分布太远。可以通过在目标函数中添加行为克隆 [101] 损失项来实现。
- **不确定性估计**：估计值函数的不确定性，在不确定时采取保守动作。可以使用集成方法或贝叶斯神经网络来量化不确定性。
- **重要性采样**：使用重要性采样来校正分布偏移，但可能面临方差过大的问题。

在 VTOL 转换控制中，离线 RL 可以从仿真数据、人类飞行员演示或传统控制器记录的数据中学习。研究表明，结合高质量的仿真数据和少量的真实飞行数据，离线 RL 可以学习到有效的转换策略。特别是，使用传统控制器的记录数据作为初始数据集，然后用离线 RL 进行改进，是一种安全有效的开发流程。

5.2.3 在线自适应方法

在线自适应方法可以在飞行过程中实时调整控制策略，应对未建模动力学和外部扰动。这对于 VTOL 飞行器特别重要，因为其动力学在转换过程中发生剧烈变化，且容易受到风扰等外部因素的影响。在线自适应使得飞行器能够持续学习和改进，补偿 Sim-to-Real 鸿沟，适应环境的变化。

自适应控制架构：在线自适应通常采用以下架构：(1) 基础策略：在仿真中预训练的策略，提供基本的控制能力；(2) 适应模块：根据实时观测估计环境参数或策略参数的变化；(3) 在线更新：根据估计结果调整策略。这种架构保证了即使适应失败，基础策略仍能提供基本的安全保证。

参数自适应：一种简单有效的方法是在线调整策略网络的某些参数。例如，可以使用梯度下降根据实时跟踪误差调整网络的最后几层参数。这种方法计算成本低，适合实时应用。更激进的方法可以调整更多参数，但需要更多的计算资源和更谨慎的安全检查。

元学习适应：更高级的方法是使用元学习训练一个能够快速适应的策略。在训练阶段，策略学习如何从短时间的历史数据中提取环境信息并调整行为；在部署阶段，策略可以根据最近几秒钟的飞行数据快速适应新的风况或动力学变化。元学习适应的速度通常比传统在线学习快一个数量级。

安全约束：在线自适应需要考虑安全性。自适应过程不应导致策略性能暂时下降或产生危险行为。安全自适应方法限制每次更新的幅度，或只在安全状态下进行更新。贝叶斯方法可以量化适应过程中的不确定性，在不确定时采取保守动作。

适应与遗忘的平衡：在线自适应面临灾难性遗忘的风险，即适应新环境时可能遗忘旧环境中学到的知识。解决这一问题的方法包括使用经验回放保存历史数据、正则化方法限制参数变化、或者使用模块化架构隔离不同环境的知识。

计算资源管理：在线自适应需要在机载计算机上实时运行，计算资源有限。轻量级的适应算法（如仅适应最后几层网络参数、使用低维参数化表示）可以在保证适应效果的同时满足实时性要求。

收敛性保证：在线自适应算法的收敛性是一个重要理论问题。需要确保适应过程能够收敛到正确的参数估计，且收敛速度足够快。李雅普诺夫稳定性理论可以用于分析自适应系统的稳定性。

多环境适应：在实际应用中，飞行器可能需要在多种环境之间切换（如室内/室外、有风/无风）。在线自适应策略需要能够快速识别当前环境并切换到相应的控制模式，或者在多个环境模型之间进行插值。

5.3 悬停与前飞控制

5.3.1 悬停稳定性控制

在悬停模式，VTOL 飞行器类似于多旋翼，依靠旋翼产生升力维持悬停。然而，VTOL 的构型通常不如多旋翼优化，可能存在控制冗余度较低、地面效应更强、气动干扰更复杂等问题。在悬停模式下，VTOL 需要抵抗风扰和其他扰动保持稳定，这是实现精确任务（如垂直起降、空中悬停观察、精确吊装）的基础。

悬停动力学：VTOL 在悬停模式下的动力学与多旋翼类似，但可能存在额外的复杂性。例如，倾转旋翼在悬停时旋翼产生下洗气流，与机翼相互作用产生复杂的干扰效应，这种干扰可能导致非对称的升力分布；尾座式 VTOL 的重心位置可能不利于悬停稳定性，需要更精细的控制。

风扰抵抗：悬停时对风扰特别敏感，因为飞行器没有前飞速度来提供气动阻尼。即使是轻微的风扰也会导致显著的位置漂移。RL 方法可以学习鲁棒悬停控制策略，通过在训练中引入各种风况（恒定风、阵风、湍流），使策略学会主动抵抗风扰。研究表明，基于 RL 的悬停控制在强风条件下的性能优于传统 PID 控制器，能够将位置漂移降低 30-50%。

位置保持精度：某些任务（如空中加油、精确吊装、建筑检查）要求极高的位置保持精度（厘米级）。RL 可以通过精细的奖励设计学习高精度的位置控制策略，同时考虑执行器饱和、控制延迟、传感器噪声等实际约束。高精度控制通常需要融合多种传感器信息，如 GPS、视觉、激光雷达等。

5.3.2 高效前飞控制

在前飞模式，VTOL 飞行器需要优化气动效率，实现长航程飞行。与悬停模式不同，前飞模式下机翼产生升力，旋翼主要用于提供推力，气动效率显著提高。然而，VTOL 的构型（如倾转旋翼、尾座式）通常不如纯固定翼优化，需要精细的控制策略来最大化航程。

气动效率优化：前飞效率通常用升阻比 (Lift-to-Drag Ratio) 衡量。RL 方法可以学习最小阻力飞行策略，通过优化攻角、侧滑角、滚转角等参数，在给定速度下最大化升阻比，延长航程。奖励函数可以设计为：

$$r = -\lambda_1 \cdot D + \lambda_2 \cdot V_{ground} \quad (37)$$

其中 D 是阻力， V_{ground} 是地速。策略需要在最小化阻力和最大化速度之间取得平衡。

过渡飞行优化：从悬停到前飞的过渡过程中，飞行器需要同时调整姿态、速度、高度。RL 可以学习最优过渡轨迹，最小化过渡时间或能耗。研究表明，优化的过渡轨迹可以显著降低过渡过程中的能耗，延长总航程。

风扰适应：前飞时对风扰的敏感度与悬停时不同。顺风可以增加地速但可能影响稳定性，逆风降低地速但增加控制余度。RL 策略可以学习根据风况调整飞行参数，如选择最优高度层、调整航向以利用侧风等。

5.4 控制分配与冗余管理

VTOL 飞行器通常具有冗余执行器，包括旋翼、舵面和推进器。控制分配问题需要协调这些执行器产生期望的力和力矩。

RL 方法可以学习最优控制分配策略，在考虑执行器约束的前提下最小化能耗。

5.5 方法对比与局限性

VTOL 控制是当前 RL 飞行控制研究的新兴热点，但也面临诸多挑战：

模型不确定性：VTOL 在不同飞行阶段的气动模型差异大。悬停时主要受旋翼气动影响，前飞时主要受机翼气动影响，过渡阶段则是两者的复杂耦合。这种时变、非线性的动力学特性难以用单一模型准确描述，给控制设计带来挑战。

控制分配：多执行器的协调控制问题复杂。VTOL 通常具有多个控制输入（旋翼、舵面、推进器），需要协调这些执行器产生期望的力和力矩。控制分配需要考虑执行器饱和、故障、效率变化等约束，是一个多目标优化问题。

故障容错：单发失效等故障情况下的安全控制困难。VTOL 的故障容错能力取决于具体构型，某些构型在单发失效后可能失去控制能力。设计故障容错控制策略需要考虑各种故障模式，确保在故障情况下能够安全着陆或返航。

安全性要求：模态转换失败可能导致灾难性后果。模态转换是 VTOL 飞行中最危险的阶段，转换失败可能导致失控坠毁。因此，VTOL 控制策略必须经过严格的安全验证，确保在各种情况下都能完成安全转换。

6 Sim-to-Real 迁移

Sim-to-Real 迁移是强化学习从学术研究走向实际应用的关键环节。由于真实世界训练的高成本和风险，大多数 RL 研究在仿真环境中进行。然而，仿真与现实之间存在动力学差异，导致在仿真中训练的策略在真实系统上性能下降。本节系统综述 Sim-to-Real 迁移的主要技术路线。

6.1 仿真到现实鸿沟

仿真到现实鸿沟 (Reality Gap) 是指仿真环境与真实环境之间的差异。这些差异主要来自以下几个方面：

动力学差异：仿真模型无法完全捕捉真实系统的动力学特性，特别是复杂的气动效应和机械摩擦。

感知差异：仿真中的传感器模型简化，无法完全模拟真实传感器的噪声、延迟和故障模式。

执行器差异：真实执行器存在死区、饱和和响应延迟，这些在仿真中难以精确建模。

表 4: 主流飞行仿真器对比

仿真器	物理精度	视觉渲染	计算效率	社区支持
Gazebo [92]	中	中	高	高
AirSim [103]	高	高	中	高
PyBullet [87]	中	低	高	中
Isaac Gym	高	中	很高	中
Flightmare [12]	高	高	高	低

表 4对比了主流飞行仿真器的特性。不同仿真器在物理精度、视觉渲染、计算效率等方面各有优劣，选择合适的仿真器是 Sim-to-Real 成功的第一步。

图 3展示了 Sim-to-Real 迁移的完整技术路线。从仿真训练到真实部署，需要经过多个技术环节，包括域随机化、域适应、系统辨识等。每个环节都有多种方法可选，需要根据具体应用场景进行选择 and 组合。

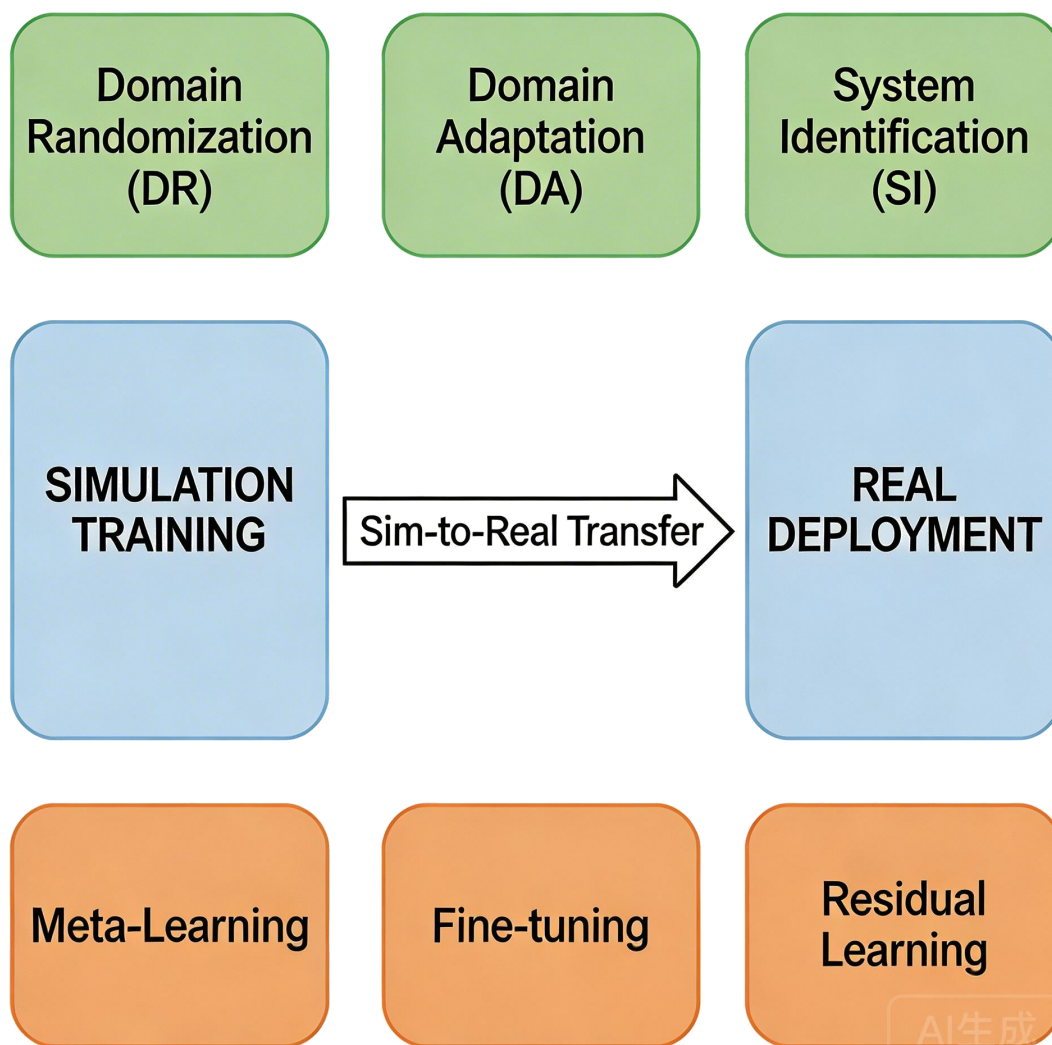


图 3: Sim-to-Real 技术路线图

6.2 域随机化技术

域随机化 (Domain Randomization, DR) 是最常用的 Sim-to-Real 技术之一。其核心思想是在仿真中随机化环境参数，训练对参数变化鲁棒的策略。

6.2.1 参数随机化策略

域随机化的核心在于选择合适的随机化参数和随机化范围。参数选择需要平衡两个目标：足够的随机化以覆盖真实环境的变化，同时避免过度随机化导致学习困难。

常见的随机化参数包括：

动力学参数：质量、惯性矩阵、推力系数、阻力系数、力矩系数等。这些参数直接影响飞行器的运动响应，是域随机化的重点。例如，四旋翼的质量可以在标称值的 $\pm 20\%$ 范围内随机化，以模拟不同载荷配置。

几何参数：旋翼半径、翼展、重心位置、安装角等。几何参数的变化会影响气动特性和控制效率。在固定翼控制中，翼展和弦长的随机化可以模拟不同机型或结冰等效应。

环境参数：重力加速度、空气密度、风速、风向、湍流强度等。环境参数对飞行性能有显著影响，特别是在室外飞行时。风速随机化可以训练出对风扰具有鲁棒性的策略。

噪声参数：传感器噪声 (陀螺仪漂移、加速度计噪声、GPS 误差)、执行器延迟和噪声、控制指令更新率等。这些参数影响感知-控制回路的性能。

随机化分布：参数通常从均匀分布或截断高斯分布中采样。均匀分布保证在整个范围内都有采样，而高斯分布更集中在标称值附近。选择哪种分布取决于对真实环境变化的先验知识。

课程随机化：从窄随机化范围开始，随着训练进行逐渐扩大范围。这种课程学习策略可以提高学习效率，避免早期训练时过度困难的随机化环境阻碍学习。

对抗随机化：使用对抗学习自动选择随机化参数，使得策略在最困难的随机化环境中也能表现良好。这种方法可以减少人工设计随机化范围的工作量。

随机化范围的选择是关键。范围过小无法覆盖真实差异，范围过大则增加训练难度。通常需要根据先验知识或初步实验来确定合理的随机化范围。

6.2.2 动力学随机化

动力学随机化 (Dynamics Randomization) 是域随机化的核心组成部分，通过在仿真中改变动力学参数来训练鲁棒策略。研究表明，适当程度的动力学随机化可以显著提高策略的泛化能力，是实现 Sim-to-Real 迁移的关键技术。

随机化参数选择：动力学随机化的参数包括质量、惯性矩阵、推力系数、阻力系数、力矩系数、电机响应时间常数等。这些参数直接影响飞行器的运动响应，是仿真与现实差异的主要来源。

随机化范围设计：随机化范围的选择需要平衡。范围过小无法覆盖真实差异，导致迁移失败；范围过大则增加训练难度，甚至使任务无法完成。通常采用渐进式方法：从

较小范围开始，根据验证结果逐步扩大范围，直到策略在真实系统上表现良好。

参数相关性：某些动力学参数之间存在物理相关性。例如，质量增加通常伴随着惯性增加。在随机化时保持这些相关性可以提高训练效果。可以通过从物理合理的参数分布中采样来实现这一点。

多保真度训练：使用不同保真度的仿真模型进行训练。低保真模型计算快但精度差，适合早期快速探索；高保真模型计算慢但精度高，适合后期精细调整。这种多保真度方法可以提高训练效率。

6.2.3 视觉随机化

对于视觉控制任务，视觉随机化尤为重要，因为仿真图像与真实图像之间存在显著的视觉差异（称为“视觉现实鸿沟”）。常见的视觉随机化包括：

光照变化：随机化光源位置、强度、颜色温度，模拟不同时间（早晨、正午、傍晚）、不同天气（晴天、阴天）的光照条件。

纹理变化：改变场景纹理，如地面材质（草地、混凝土、泥土）、建筑物外观、障碍物纹理等。可以使用纹理合成技术生成多样化的纹理。

相机参数变化：随机化相机内参（焦距、主点）、外参（安装角度、位置）、畸变参数，模拟不同相机型号和安装方式。

图像噪声和畸变：添加高斯噪声、椒盐噪声、运动模糊、颜色抖动等，模拟真实相机的噪声特性。

域随机化与域适应结合：视觉随机化可以与域适应技术结合，进一步提高视觉策略的迁移能力。随机化提供多样化的训练数据，域适应对齐仿真和真实的特征分布。这种组合方法在视觉导航、目标跟踪等任务中表现出色。

渲染引擎差异：不同仿真器使用不同的渲染引擎（如 Gazebo 使用 OGRE，AirSim 使用 Unreal Engine），产生的图像风格不同。在多种渲染引擎上训练可以提高策略对不同视觉风格的适应能力。

动态场景处理：标准视觉随机化假设场景是静态的，但实际飞行中场景可能动态变化（如移动障碍物、变化的阴影）。动态视觉随机化需要考虑时序一致性，确保随机化后的视频序列仍然合理。

6.3 域适应方法

域适应 (Domain Adaptation) 通过对抗训练或其他方法对齐仿真和真实的特征分布。

6.3.1 对抗域适应

对抗域适应 (Adversarial Domain Adaptation) 使用对抗网络来区分仿真特征和现实特征，同时训练特征提取器使其无法被区分。这样学习到的特征表示对域差异具有不变

性，从而实现从仿真到现实的迁移。

对抗训练机制：对抗域适应包含两个主要组件：特征提取器 G 和域分类器 D 。特征提取器将输入（状态或观测）映射到特征空间，域分类器试图根据特征判断输入来自仿真域还是现实域。训练过程中，域分类器最大化分类准确率，而特征提取器最小化分类准确率（即让域分类器无法区分）。这种对抗训练使得特征提取器学习到域不变的特征表示。

飞行控制应用：在飞行控制中，对抗域适应可以用于视觉感知任务。例如，将仿真中训练的视觉导航策略迁移到真实环境时，对抗训练可以使特征提取器对仿真图像和真实图像之间的差异（如光照、纹理、噪声）具有不变性，只关注对导航任务重要的特征（如障碍物边缘、路径标记）。

与 RL 结合：对抗域适应可以与强化学习结合，在策略训练的同时进行域适应。这种端到端的训练方式可以学习到对 Sim-to-Real 迁移最优的策略和特征表示。联合训练使得策略和特征提取器能够相互适应，共同优化迁移性能。

梯度反转层：实现对抗域适应的关键技术是梯度反转层（Gradient Reversal Layer, GRL）。在前向传播时，GRL 作为恒等映射；在反向传播时，GRL 反转梯度符号。这使得特征提取器在优化分类损失时，实际上是在最小化域分类器的性能，即学习域不变的特征。

多源域适应：在实际应用中，可能需要从多个仿真环境迁移到真实环境。多源域适应扩展了对抗域适应，处理多个源域的情况。特征提取器学习对所有源域和目标任务都有效的特征表示。

6.3.2 元学习方法

元学习（Meta-Learning），也称为“学会学习”（Learning to Learn），训练能够快速适应新环境的策略。与传统的机器学习方法需要大量数据来训练特定任务的模型不同，元学习旨在学习一个能够快速适应各种相关任务的通用模型。

模型无关元学习（Model-Agnostic Meta-Learning, MAML [9]）是代表性的元学习方法。MAML 学习一个好的参数初始化，使得少量梯度更新即可适应新任务。具体来说，MAML 在训练阶段模拟适应过程：对于每个任务，从初始化参数开始进行几次梯度更新，然后评估更新后的性能；优化目标是找到一个好的初始化，使得经过几次更新后所有任务的平均性能最优。

在飞行控制中，元学习可以用于快速适应不同的飞行器配置或环境条件。例如，训练一个元策略，使其能够快速适应不同质量、不同气动特性的飞行器，或适应不同的风况。这对于商业应用特别有价值，因为同一控制策略可能需要部署到大量不同的飞行器上。

在线元学习：更进一步的，元学习可以与在线适应结合。策略在部署后持续进行元学习，不断积累适应经验，使得后续的适应越来越快。这种“终身元学习”能力使得系统能够持续改进，适应环境的变化。

6.3.3 在线适应策略

在线适应策略在部署后持续学习，根据真实系统的反馈调整策略。这与传统的离线训练-在线部署模式不同，在线适应允许策略在真实环境中不断改进，补偿 Sim-to-Real 鸿沟。

挑战：在线适应面临几个关键挑战：(1) 探索-利用困境：在线学习需要探索以收集数据，但探索可能导致性能下降甚至危险行为；(2) 灾难性遗忘：适应新环境时可能遗忘旧环境中学到的知识；(3) 样本效率：真实环境的样本昂贵，需要高效的在线学习算法。

安全在线学习：为了解决安全挑战，研究人员提出了安全在线学习方法。这些方法在适应过程中保证策略不会显著偏离已知安全的策略，或限制探索范围在安全区域内。例如，使用信任区域方法限制每次更新的幅度，或使用安全层确保动作满足安全约束。

贝叶斯在线适应：贝叶斯方法可以量化适应过程中的不确定性，在不确定时采取保守动作。例如，使用贝叶斯神经网络或高斯过程来建模环境动力学，根据不确定性调整探索程度。

计算资源管理：在线自适应需要在机载计算机上实时运行，计算资源有限。轻量级的适应算法（如仅适应最后几层网络参数、使用低维参数化表示）可以在保证适应效果的同时满足实时性要求。

收敛性保证：在线自适应算法的收敛性是一个重要理论问题。需要确保适应过程能够收敛到正确的参数估计，且收敛速度足够快。李雅普诺夫稳定性理论可以用于分析自适应系统的稳定性。

多环境适应：在实际应用中，飞行器可能需要在多种环境之间切换（如室内/室外、有风/无风）。在线自适应策略需要能够快速识别当前环境并切换到相应的控制模式，或者在多个环境模型之间进行插值。

6.4 系统辨识与模型校准

系统辨识通过实验数据估计仿真模型的参数，使仿真模型更接近真实系统。

6.4.1 参数估计方法

系统辨识通过实验数据估计仿真模型的参数，使仿真模型更接近真实系统。常用的参数估计方法包括：

最小二乘法 (Least Squares): 最简单的参数估计方法，最小化预测输出与实际输出之间的平方误差。对于线性系统，最小二乘法有解析解；对于非线性系统，可以使用迭代最小二乘法。

最大似然估计 (Maximum Likelihood Estimation, MLE): 假设测量噪声服从某种概率分布（如高斯分布），寻找使观测数据似然最大的参数。MLE 在统计意义上是最优的，但计算复杂度较高。

贝叶斯方法：将参数视为随机变量，根据观测数据更新参数的后验分布。贝叶斯方法可以量化参数估计的不确定性，对于后续的策略优化和安全分析有价值。

辨识机动设计：对于飞行系统，通常需要设计专门的辨识机动来激发系统动力学。例如，通过施加扫频信号或伪随机序列来激发不同频率的响应。辨识机动需要在保证安全的前提下充分激发系统动态。

递归最小二乘法：递归最小二乘法 (Recursive Least Squares, RLS) 是在线参数估计的常用方法，可以在每个时间步根据新数据更新参数估计，无需存储所有历史数据。RLS 计算效率高，适合实时应用。

参数可辨识性：并非所有参数都能从给定数据中准确估计。参数可辨识性分析可以确定哪些参数可以被估计，以及需要什么样的输入信号才能准确估计这些参数。对于不可辨识的参数，需要固定其值或引入先验知识。

6.4.2 残差学习

残差学习 (Residual Learning) 训练神经网络来预测仿真模型与真实系统之间的差异。设仿真模型的预测为 $\hat{x}_{sim}$ ，真实系统的观测为 x_{real} ，残差模型 f_{res} 学习预测差异：

$$x_{real} \approx \hat{x}_{sim} + f_{res}(s, a) \quad (38)$$

这个残差模型可以用于补偿仿真误差，提高策略的迁移性能。在训练阶段，策略在补偿后的仿真环境中训练；在部署阶段，残差模型根据实时数据不断更新，持续提高仿真保真度。

在线残差学习：残差模型可以在线学习，根据真实飞行数据持续更新。这种在线学习使得仿真模型能够跟踪真实系统的变化 (如磨损、损伤)，保持高保真度。

残差模型架构：残差模型可以采用多种神经网络架构，如多层感知机 (MLP)、循环神经网络 [90](RNN) 处理时序依赖、或者物理信息神经网络 (PINN) 融入物理约束。架构选择取决于残差的特性和计算资源限制。

误差补偿策略：残差模型预测的误差可以用于多种补偿策略：前馈补偿 (在控制输入中加入补偿项)、反馈补偿 (调整控制器增益)、或者重规划 (根据补偿后的模型重新规划轨迹)。不同策略适用于不同的应用场景。

6.4.3 混合模型方法

混合模型方法 (Hybrid Model) 结合物理模型和数据驱动模型。物理模型捕捉主要的动力学特性 (如刚体动力学)，神经网络学习剩余的未建模动力学 (如复杂气动效应、执行器非线性)。

架构设计：混合模型可以表示为：

$$\dot{x} = f_{physics}(x, u) + f_{nn}(x, u) \quad (39)$$

其中 $f_{physics}$ 是物理模型， f_{nn} 是神经网络模型。这种分解利用了物理先验，减少了神经网络需要学习的内容，提高了泛化能力和样本效率。

训练策略：混合模型的训练可以采用多阶段方法：首先根据物理原理初始化物理模型参数，然后固定物理模型训练神经网络学习残差，最后联合微调所有参数。

物理约束嵌入：在混合模型中，可以显式嵌入物理约束（如能量守恒、动量守恒），确保模型满足基本物理规律。物理信息神经网络 (PINN) 通过在损失函数中加入物理方程的残差项，实现数据驱动与物理约束的结合。

可解释性优势：混合模型相比纯数据驱动模型具有更好的可解释性。物理模型部分提供可解释的物理意义，神经网络部分解释未建模效应。这种分解便于调试、验证和认证。

6.5 成功案例分析

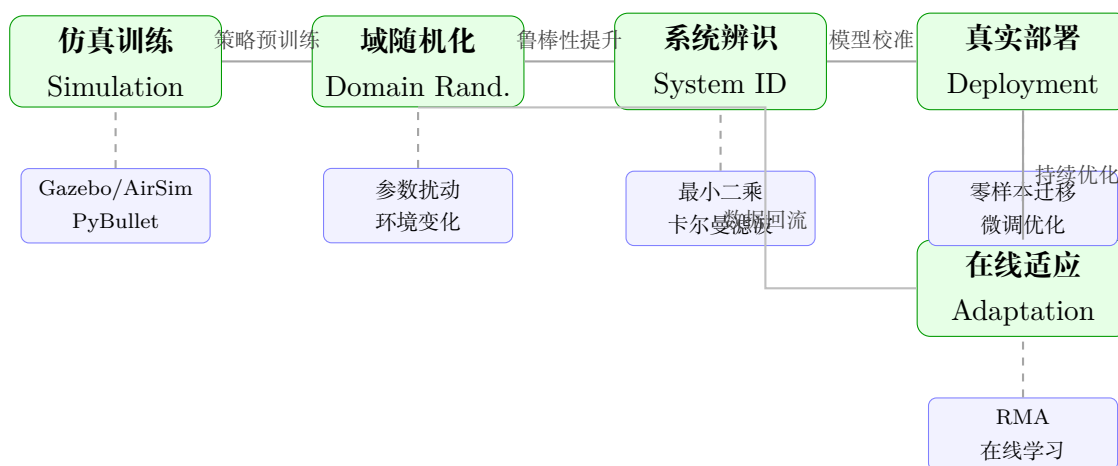


图 4: Sim-to-Real 技术路线图

图 4 展示了 Sim-to-Real 迁移的完整技术路线。下面分析几个成功的案例。

6.5.1 ETH Zurich 敏捷飞行

ETH Zurich 的研究团队在 2021 年实现了高速敏捷飞行的 Sim-to-Real 迁移，这是该领域的里程碑工作。他们使用大规模的域随机化，在仿真中以超过 100km/h 的速度训练，成功迁移到真实无人机，在复杂环境中实现了与人类世界冠军级飞行员相媲美的性能。

技术突破：该工作的关键突破在于证明了仅通过仿真训练就能达到专家级性能，无需在真实系统上进行昂贵的训练。这为强化学习在实际飞行系统中的应用开辟了道路。

关键技术包括：

- **大规模并行仿真训练：**使用数千个仿真实例并行训练，在数小时内完成数月的训练量。并行训练不仅加速了学习，还通过多样化的仿真实例提高了策略的泛化能

力。

- **系统性的域随机化策略**：不仅随机化动力学参数，还随机化视觉环境（光照、纹理、障碍物形状）、传感器参数（相机内参、噪声模型）等。这种全面的随机化使得策略对各种环境变化具有鲁棒性。
- **感知-动作联合学习**：策略网络同时处理高维视觉输入（相机图像）和低维惯性数据（IMU），学习端到端的感知-动作映射。这种联合学习使得策略能够直接从原始传感器数据学习，无需人工设计的特征提取。

实验验证：在真实世界的竞赛中，该方法与人类世界冠军级飞行员进行了对比。结果显示，在多个赛道上，RL 策略的飞行时间与人类专家相当，在某些复杂赛道上甚至超越了人类。更重要的是，RL 策略展现出更好的一致性，在不同尝试中飞行时间的变化更小。

6.5.2 MIT RMA [22] 算法

MIT 的 Rapid Motor Adaptation (RMA) 算法虽然最初是为四足机器人开发的，但其核心思想对飞行控制同样适用。RMA 实现了快速适应不同地形和负载的行走控制，其核心思想是训练一个基础策略和一个适应模块，适应模块根据历史观测估计环境参数并调整策略。

架构设计：RMA 包含两个主要组件：(1) 基础策略 π_{base} ，在多样化的随机化环境中训练，学习通用的控制技能；(2) 适应模块 ϕ ，根据历史观测序列估计当前环境参数，并生成适应后的策略。这种分离设计使得系统能够快速适应新环境，而无需重新训练整个策略。数学上，适应后的策略可以表示为 $\pi_{adapted}(a|s) = \pi_{base}(a|s, \phi(h))$ ，其中 h 是历史观测序列。

训练过程：RMA 的训练分为两个阶段。第一阶段，在多样化的随机化环境中训练基础策略，使其学习通用的运动技能。第二阶段，冻结基础策略，训练适应模块根据历史观测预测环境参数。训练目标是使适应后的策略在各种环境中都能表现良好。

适应速度：RMA 的突出特点是适应速度快。传统的在线适应方法可能需要数十秒甚至数分钟才能适应新环境，而 RMA 可以在几秒内完成适应。这是因为适应模块只需要进行前向推理，无需梯度更新。这对于飞行控制特别重要，因为飞行器动力学可能在短时间内发生显著变化（如进入风切变区域）。

飞行控制应用：RMA 的思想可以应用于飞行控制中的在线适应。例如，当飞行器进入不同的风况或载荷发生变化时，适应模块可以快速估计这些变化并调整控制策略，保持稳定飞行。研究表明，基于 RMA 思想的飞行控制器可以在数秒内适应新的风况，显著优于固定策略。

6.5.3 Neural-Fly 风扰适应

Neural-Fly 方法专门设计了用于风扰适应的在线学习算法，由 Caltech 的研究团队于 2022 年提出。它使用元学习训练一个能够快速适应风况变化的策略，在数秒内即可适应新的风况，为无人机在复杂风环境下的飞行提供了有效的解决方案。

问题背景：风扰是无人机飞行中的主要挑战之一，特别是在城市环境、山区或近地面飞行时。风场在空间和时间上变化，固定的控制策略难以应对。传统的风速估计方法需要额外的传感器 (如风速计、风标)，增加了系统复杂度、重量和成本。此外，机载风速测量通常只能测量局部风速，无法反映整个风场分布。

方法核心：Neural-Fly 的核心思想是利用飞行器自身的动力学响应来隐式估计风场。当飞行器受到风扰时，其运动状态 (加速度、角速度) 会发生变化，这些变化包含了风场信息。通过训练神经网络从状态序列中提取风场特征，可以在无需额外传感器的情况下估计风况。这种方法利用了飞行器本身作为”风传感器”，通过其动力学响应反推风场。

网络架构：Neural-Fly 采用编码器-解码器架构。编码器处理历史状态序列，提取风场特征；解码器根据当前状态和风场特征生成控制指令。这种架构使得策略能够同时考虑当前状态和历史信息，做出更准确的控制决策。

元学习适应：Neural-Fly 使用模型无关元学习 (MAML) 训练适应模块，使其能够从短时间 (数秒) 的观测中快速适应新的风况。在元训练阶段，策略学习如何从少量数据中提取风场信息；在部署阶段，策略可以根据最近几秒钟的飞行数据快速估计当前风况。这种快速适应能力使得飞行器能够在穿越风切变区域或城市峡谷时保持稳定性。

实验验证：在真实世界的实验中，Neural-Fly 在强风条件下 (风速超过 10m/s) 显著优于固定策略，接近使用精确风速测量的理想控制器性能。实验在 Caltech 的室外飞行场地进行，使用了定制的风场生成设备来模拟各种风况。结果表明，Neural-Fly 能够将位置跟踪误差降低 50% 以上，特别是在阵风条件下表现出色。

6.6 技术对比与最佳实践

表 5: Sim-to-Real 方法对比

方法	真实数据需求	计算成本	迁移成功率	适用场景
域随机化	无	中	70-85%	通用场景
域适应	少量	高	80-90%	大差距场景
系统辨识	中等	中	85-95%	精确控制
元学习	极少	很高	75-85%	快速部署

表 5对比了不同 Sim-to-Real 方法的特点。在实际应用中，通常需要组合多种方法来获得最佳效果。

最佳实践建议：

1. 选择高保真度仿真器，尽可能准确地建模真实系统。
2. 实施系统辨识，校准仿真模型参数。
3. 使用域随机化训练鲁棒策略。
4. 如有真实数据，使用域适应进一步提升性能。
5. 部署后实施在线适应，持续优化策略。

7 挑战与展望

尽管强化学习在飞行控制领域取得了显著进展，但仍面临诸多挑战。本节总结当前的主要挑战，并展望未来的研究方向。

7.1 当前技术挑战

7.1.1 样本效率问题

强化学习通常需要大量的交互样本来学习有效的策略。对于飞行控制这种高风险、高成本的领域，样本效率是关键瓶颈。在真实飞行器上收集数据不仅成本高昂，而且存在安全风险，这使得样本效率成为制约 RL 实际应用的主要因素。

样本效率现状：当前最先进的无模型 RL 算法 (如 SAC、PPO) 通常需要数百万到数千万个时间步的训练数据才能收敛。对于飞行控制任务，这意味着需要在仿真中进行数小时甚至数天的训练，或者进行大量的真实飞行试验。相比之下，人类飞行员通常只需要少量示范就能掌握基本飞行技能。

提高样本效率的可能途径包括：

- **模型-based RL：**学习环境的动力学模型，使用模型进行规划和虚拟样本生成。模型-based 方法可以显著提高样本效率，但面临模型误差和计算复杂度的挑战。在飞行控制中，可以使用物理先验 (如刚体动力学) 来约束学习到的模型，提高泛化能力。
- **模仿学习：**利用专家示范 (如人类飞行员或传统控制器) 来初始化策略，减少随机探索。行为克隆 (Behavioral Cloning) 可以直接从专家数据学习策略，而生成对抗模仿学习 (GAIL [89]) 可以通过对抗训练提取专家策略的隐含奖励函数。
- **迁移学习 [100]：**将在相关任务 (如不同飞行器、不同环境) 上学到的知识迁移到新任务。迁移学习可以显著减少新任务所需的训练数据，特别是在目标任务数据稀缺的情况下。
- **多保真度优化：**从低保真仿真 (计算成本低但精度差) 逐步过渡到高保真仿真和真实系统。这种方法可以在早期训练阶段快速探索，在后期阶段精细调整。

- **离线强化学习**：从固定的历史数据集中学习策略，无需在线交互。这对于利用现有的飞行日志数据特别有价值。

未来方向：结合多种技术的混合方法可能是解决样本效率问题的关键。例如，使用模仿学习初始化策略，然后使用模型-based RL 进行微调，最后通过少量真实飞行数据进行在线适应。这种分阶段训练策略可以充分利用不同方法的优势，在保持安全性的同时提高学习效率。此外，利用仿真到现实的渐进迁移、课程学习等技术也可以显著提高样本效率。

7.1.2 安全性与可解释性

飞行控制对安全性有极高要求，任何控制失误都可能导致飞行器坠毁和人员伤亡。然而，神经网络控制器的决策过程往往难以解释和验证，这与航空领域严格的认证要求(如 DO-178C、DO-254) 存在根本冲突。

安全强化学习 (Safe RL) 研究如何在满足安全约束的前提下进行学习。主要方法包括：

- **约束 MDP**：在优化累积奖励的同时满足安全约束。将安全要求形式化为约束条件(如状态必须在安全集合内)，使用约束优化算法求解。
- **控制屏障函数** (Control Barrier Functions, CBF)：使用控制屏障函数来保证系统状态始终在安全集合内。CBF 提供了一种系统化的方法来设计安全控制器，可以与 RL 结合使用，在保证安全的前提下优化性能。
- **可达性分析**：计算系统的可达集，验证在给定初始状态和控制器下，系统是否会进入不安全状态。Hamilton-Jacobi 可达性分析可以处理非线性系统和复杂约束。
- **安全层设计**：在 RL 策略之外添加一个安全层，当检测到不安全动作时进行干预或修正。这种分层架构可以在保持 RL 性能优势的同时提供安全保证。

可解释性研究旨在理解神经网络的决策过程。对于飞行控制，可解释性有助于：

- **调试和诊断**：当控制器出现异常行为时，可解释性工具可以帮助工程师理解原因并进行修复。
- **认证和验证**：航空认证要求能够验证控制器的正确性。可解释性技术可以生成人类可理解的控制律描述，支持认证过程。
- **信任建立**：人类操作员需要理解自主系统的行为逻辑才能建立信任。可解释性可以增强人机协作的效果。

可解释性技术包括：

- **注意力可视化**：显示策略网络在决策时关注的输入特征

- **概念激活向量 (CAV)**: 识别网络中的高层概念 (如”高度过低”、”速度过快”)
- **局部近似**: 使用线性模型或决策树局部近似神经网络的行为
- **反事实解释**: 解释”如果输入不同, 输出会如何变化”

未来方向: 开发既具有神经网络表达能力又具有可解释性的控制器架构, 如神经符号 AI、基于物理的神经网络等。这些混合架构结合了数据驱动的学习能力和物理模型的可解释性, 有望在保持高性能的同时满足认证要求。此外, 形式化验证方法的发展也为神经网络控制器的安全性证明提供了新的工具。

7.1.3 实时性要求

飞行控制系统通常需要数百赫兹的控制频率 (如姿态控制环通常需要 500Hz-1kHz), 而深度神经网络的推理计算必须在严格的时间限制内完成。这对于资源受限的机载计算机提出了巨大挑战。

实时性挑战: 深度神经网络的前向推理涉及大量矩阵乘法和激活函数计算。虽然现代深度学习框架 (如 TensorRT、ONNX Runtime) 进行了大量优化, 但在嵌入式平台上实现高频率推理仍然困难。例如, 一个包含数百万参数的策略网络在 ARM Cortex-A53 处理器上可能需要数毫秒才能完成一次推理, 无法满足 1kHz 的控制频率要求。

满足实时性要求的技术包括:

- **网络压缩**: 剪枝 (Pruning) 移除网络中不重要的连接或神经元, 减少参数量和计算量; 量化 (Quantization) 将浮点数权重转换为低精度整数 (如 INT8), 减少内存占用和计算复杂度; 知识蒸馏 (Knowledge Distillation) 训练一个小网络来模仿大网络的行为, 在保持性能的同时减小模型规模。
- **专用硬件**: 使用 GPU (如 NVIDIA Jetson 系列) 进行并行计算加速; FPGA (Field-Programmable Gate Array) 可以定制硬件电路实现特定网络架构, 达到极高的推理速度; 专用 AI 芯片 (如 Google Coral、Intel Movidius) 针对神经网络推理进行了硬件级优化。
- **边缘计算**: 将计算任务卸载到边缘服务器, 机载系统只负责传感器数据采集和执行器控制。这种方法需要可靠的通信链路, 适用于有地面站支持的场景。
- **分层控制**: 高频控制 (如姿态稳定) 使用传统控制器 (如 PID), 低频控制 (如轨迹规划) 使用 RL 策略。这种分层架构减少了对 RL 推理速度的要求。
- **网络架构优化**: 使用轻量级网络架构 (如 MobileNet、ShuffleNet), 或设计专门的飞行控制网络架构, 减少计算量同时保持性能。

未来方向：神经形态计算 (Neuromorphic Computing) 是一种新兴技术，使用类脑芯片进行低功耗、高并行的神经网络计算。这种技术有望在未来为飞行控制提供高效的推理平台。神经形态芯片 (如 Intel Loihi、IBM TrueNorth) 模拟生物神经元的工作方式，具有事件驱动、稀疏计算、低功耗等特点，特别适合实时控制应用。神经形态计算的另一个优势是天然的时序处理能力，可以高效处理飞行控制中的时间序列数据。随着神经形态硬件的成熟，预计未来可以实现超低功耗的机载智能推理。

7.1.4 标准化与评估

目前 RL 飞行控制领域缺乏统一的评估标准和基准测试。不同研究使用不同的仿真环境、任务定义、评估指标，使得方法之间的比较变得困难，也阻碍了技术的快速进步。建立标准化评估框架是推动该领域发展的关键。

当前问题：

- **任务定义不一致：**不同研究中的“轨迹跟踪”任务可能有不同的轨迹形状、速度要求、精度标准
- **评估指标多样：**有的研究使用跟踪误差，有的使用成功率，有的使用能耗，难以横向比较
- **仿真环境差异：**Gazebo、AirSim、PyBullet 等仿真器的物理精度不同，同一策略在不同仿真器上表现可能差异很大
- **真实实验不可复现：**真实飞行实验受环境条件 (风、温度、光照) 影响，难以精确复现

建立标准化评估框架需要：

- **定义标准的任务和指标：**建立任务库，定义标准任务 (如悬停、轨迹跟踪、着陆) 的详细规格；定义标准评估指标，如跟踪误差 (RMS、最大误差)、成功率、能耗、控制平滑性等
- **提供公开的仿真环境和数据集：**建立开源的飞行控制仿真平台，包含标准化的飞行器模型、环境模型、任务场景；提供公开数据集，包括专家示范数据、真实飞行日志等
- **组织竞赛和挑战赛：**定期举办飞行控制挑战赛，提供公平的比较平台。例如，IROS、ICRA 等会议举办的无人机竞赛，吸引了全球研究团队参与
- **基准算法：**提供标准基准算法的实现 (如 PID、LQR、MPC、PPO、SAC)，作为比较的基准线

- **Sim-to-Real 评估协议**：建立标准的 Sim-to-Real 评估流程，包括仿真训练、域随机化、真实迁移、性能评估等环节

已有努力：一些研究项目已经开始建立标准化框架。例如，Gym-PyBullet-Drones 提供了标准化的多旋翼仿真环境，支持多种飞行器模型和任务场景；Agile Autonomy Dataset 提供了高速飞行的专家示范数据，包含丰富的传感器记录和专家控制指令；Flightmare 仿真器专注于高保真度和计算效率的平衡，支持视觉渲染和物理仿真。这些开源资源为研究人员提供了共同的基准，促进了方法的比较和复现。此外，一些竞赛如 IROS 和 ICRA 的无人机挑战赛也推动了标准化评估的发展。

7.2 新兴技术趋势

7.2.1 与大语言模型结合

大语言模型 (Large Language Models, LLM) 如 GPT-4、Claude 等展现了强大的推理、规划和知识处理能力。将 LLM 与强化学习结合，有望实现更智能、更灵活的飞行控制系统，这是当前人工智能领域的前沿研究方向。

高层任务规划：LLM 可以将高层自然语言指令（如“搜索目标区域并返回基地”）分解为可执行的子任务序列（如“起飞 → 飞往区域 → 执行搜索模式 → 返航 → 着陆”）。每个子任务可以对应一个特定的 RL 策略或传统控制器。这种分层架构结合了 LLM 的语义理解能力和 RL 的精细控制能力，使得非专业人员也能通过自然语言控制复杂的无人机系统。

常识推理：LLM 蕴含丰富的世界知识，可以帮助处理训练时未遇到的意外情况。例如，当飞行器遇到未在训练数据中出现的故障模式时，LLM 可以基于其对飞行器原理的理解提供应急建议；当遇到恶劣天气时，LLM 可以建议改变航线或推迟任务。

人机交互：通过自然语言与人类操作员交流，提高系统的可用性和可接受性。操作员可以用自然语言描述任务目标、约束条件或紧急情况，系统通过 LLM 理解意图并调整控制策略。这种人机协作模式在应急救援、军事指挥等场景中特别有价值。

代码生成与策略优化：LLM 可以生成或修改 RL 的奖励函数、网络架构或超参数配置，辅助算法设计。研究表明，LLM 可以根据任务描述自动生成合理的奖励函数，减少人工设计的工作量。更进一步，LLM 可以分析策略的失败案例，提出改进建议。

挑战与局限：LLM 与 RL 的结合也面临挑战，包括计算资源需求高（大模型需要 GPU 支持）、响应延迟大（推理时间较长）、输出不确定性（LLM 可能产生幻觉）等。在实时性要求高的飞行控制中，需要设计高效的推理架构或采用边缘部署方案。此外，LLM 的安全性和对齐问题也需要关注，确保其输出不会导致危险行为。

7.2.2 神经辐射场应用

神经辐射场 (Neural Radiance Fields, NeRF) 是一种从二维图像学习三维场景表示的深度学习技术。NeRF 通过神经网络隐式地表示三维场景的体积密度和颜色，可以从

任意视角渲染出逼真的图像。这一技术为飞行控制中的环境感知和导航提供了新的可能性。

三维环境重建：传统的三维重建方法（如 SLAM）通常产生稀疏的点云或网格表示，而 NeRF 可以产生连续、高保真的三维场景表示。对于飞行控制，这意味着可以从无人机采集的图像构建详细的三维地图，用于路径规划、避障、着陆点选择等任务。NeRF 表示的连续性特别适合规划平滑的飞行轨迹。

视觉导航：在 NeRF 表示的空间中进行视觉伺服控制是一个新兴研究方向。传统视觉导航需要显式的特征提取和匹配，而 NeRF-based 方法可以直接在神经表示中进行优化。例如，可以优化相机位姿使得渲染图像与目标图像匹配，实现精确的相对定位。这种方法在 GPS 拒止环境（如室内、城市峡谷）中特别有价值。

仿真增强：NeRF 可以从真实图像生成高保真仿真环境。传统的仿真环境需要人工建模，耗时且难以保证真实性。使用 NeRF，可以从真实场景的照片快速生成可渲染的三维环境，用于训练视觉导航策略。这种真实感仿真有助于提高 Sim-to-Real 迁移的成功率。

动态场景：标准 NeRF 假设场景是静态的，而实际飞行环境中存在动态物体（如其他飞行器、行人、车辆）。动态 NeRF 扩展可以处理动态场景，为复杂环境中的飞行控制提供支持。

计算挑战：NeRF 的渲染和训练计算成本较高，在实时飞行控制中应用面临挑战。近期研究提出了加速方法（如 Instant-NGP、MobileNeRF），显著提高了渲染速度，使得 NeRF 在实时应用中成为可能。未来的发展方向包括：轻量化 NeRF 架构设计，针对嵌入式平台优化；多分辨率 NeRF 表示，根据距离动态调整细节层次；NeRF 与 SLAM 的结合，实现实时三维重建和定位。

7.2.3 联邦学习与隐私保护

联邦学习 (Federated Learning) 是一种分布式机器学习范式，允许多个参与方在不共享原始数据的情况下协作训练模型。每个参与方在本地训练模型，只共享模型参数或梯度，而不共享敏感的原始数据。这一技术对于飞行控制领域具有重要价值，特别是在涉及多个机构、大量用户的场景中。

多机构协作：飞行控制研究涉及大学、研究机构、企业等多个参与方，每个参与方拥有自己的飞行器数据和训练经验。联邦学习使得不同机构可以共享学习成果而不泄露敏感数据（如飞行器设计参数、任务细节）。例如，多个研究机构可以协作训练一个通用的飞行控制策略，每个机构贡献自己的仿真数据或真实飞行数据，但不需要共享原始数据。

个性化适应：商业无人机用户（如物流、农业、巡检公司）希望获得针对自己特定任务和飞行器的优化控制策略，但又不愿意分享自己的飞行数据。联邦学习可以在保护用户隐私的前提下实现个性化控制策略：在云端训练一个通用基础策略，然后分发到各用户的飞行器上进行个性化微调，只将更新后的参数（而非原始数据）回传到云端。

数据异质性：不同参与方的数据分布可能存在显著差异（如不同的飞行器型号、不同的飞行环境、不同的任务类型）。联邦学习需要处理这种数据异质性问题，确保协作训练的效果。个性化联邦学习（Personalized Federated Learning）为每个参与方学习个性化的模型，同时利用其他参与方的知识。

安全与攻击：联邦学习也面临安全挑战，如恶意参与方可能通过模型更新注入后门攻击，或推断其他参与方的私有信息。安全联邦学习研究如何防御这些攻击，确保协作训练的安全性和隐私性。

边缘联邦学习：对于实时性要求高的飞行控制，可以将联邦学习与边缘计算结合。飞行器作为边缘节点，在本地进行模型训练和推理，定期与云端或其他飞行器交换模型更新，实现分布式的持续学习。这种架构特别适合大规模无人机集群场景，可以在保护隐私的前提下实现群体智能。

7.2.4 数字孪生技术

数字孪生（Digital Twin）是物理系统的虚拟副本，通过实时数据同步保持与物理系统状态一致。数字孪生不仅包含几何模型，还包含物理模型、行为模型、历史数据等，能够全面反映物理系统的特性和状态。在飞行控制领域，数字孪生技术正在快速发展，为设计、测试、运维提供了全新的工具。

在线系统辨识：数字孪生可以持续更新仿真模型以匹配真实系统。通过比较数字孪生的预测与真实飞行器的实际响应，可以在线估计系统参数（如质量、气动系数、执行器特性）的变化。这种在线辨识对于适应飞行器老化、损伤、载荷变化等情况特别有价值。更新后的数字孪生可以用于更准确的仿真预测和策略优化。

预测性维护：通过分析数字孪生中的历史数据和当前状态，可以预测飞行器部件的故障和剩余寿命。例如，通过监测电机的温度、振动、电流模式，可以预测电机轴承的磨损程度，在故障发生前安排维护。这可以显著提高飞行器的可靠性和可用性，降低维护成本。

虚拟测试：在数字孪生上测试新策略，避免真实系统的风险。这对于测试极端情况（如大攻角机动、故障情况）或新算法特别有价值。通过在数字孪生上进行充分的虚拟测试，可以筛选出有潜力的策略，只在最有希望的策略上进行昂贵的真实飞行测试。虚拟测试还可以用于探索设计空间，优化飞行器设计参数。

人机协作训练：数字孪生可以用于飞行员或操作员的训练。通过与高保真的虚拟飞行器交互，操作员可以在安全的环境中练习各种机动和应急程序。对于基于 RL 的自主系统，数字孪生提供了与真实环境相似的训练环境，有助于提高 Sim-to-Real 迁移的成功率。数字孪生还可以用于人机协作场景的训练，如操作员监督下的半自主飞行。

全生命周期管理：数字孪生贯穿飞行器的全生命周期，从设计、制造、测试、运营到退役。在设计阶段，数字孪生可以用于虚拟原型验证；在运营阶段，数字孪生支持实时监控和优化；在退役阶段，数字孪生中的历史数据可以用于分析和改进下一代设计。数字孪生的数据积累还可以用于预测性维护和剩余寿命评估。

7.3 未来研究方向

基于以上分析，我们提出以下未来研究方向：

安全强化学习：开发能够提供形式化安全保证的 RL 算法，满足航空认证要求。

多模态感知融合：融合视觉、激光雷达、IMU 等多种传感器信息，提高感知鲁棒性。

边缘智能部署：研究如何在资源受限的机载计算机上高效部署和运行深度 RL 策略。

人在回路中的智能辅助：开发能够与人类飞行员协作的半自主控制系统。

大规模集群协同：研究数百架无人机的协同控制，实现复杂的群体行为。

跨平台策略迁移：开发能够在不同飞行器平台之间迁移的控制策略。

伦理与法规框架：研究自主飞行系统的伦理问题和监管框架。

8 结论

本文系统综述了强化学习在飞行控制中的应用，涵盖多旋翼、固定翼和 VTOL 三种主要飞行平台。主要结论如下：

第一，强化学习为多旋翼控制提供了一种有效的数据驱动方法。端到端 RL 策略在姿态控制、位置跟踪等任务上可以达到或超越传统控制器的性能，同时具有更好的自适应能力和多目标优化能力。特别是，SAC 和 PPO 算法因其良好的稳定性和样本效率，成为多旋翼控制的首选算法。Sim-to-Real 迁移技术的进步使得在仿真中训练的策略能够成功部署到真实飞行器上，为实际应用奠定了基础。

第二，固定翼 RL 控制面临飞行包线约束、失速风险等独特挑战。当前研究主要集中在轨迹优化、着陆控制、编队飞行等高层任务，底层姿态控制仍主要依赖传统方法。这主要是因为固定翼的动力学更为复杂，对安全性要求更高，且控制冗余度较低。未来的研究需要开发能够处理飞行包线约束的安全强化学习方法，以及能够在极端条件下(如大攻角、失速边缘)工作的控制策略。

第三，VTOL 控制是 RL 应用的新兴热点，模态转换控制是核心难题。VTOL 结合了多旋翼和固定翼的优点，但也继承了两者的控制挑战。模态转换过程中的时变动力学、控制分配复杂性、安全性要求使得传统的控制方法难以应对。安全强化学习和离线 RL 方法在这一领域具有重要价值，可以在保证安全的前提下学习复杂的转换策略。

第四，Sim-to-Real 迁移是 RL 飞行控制实际部署的关键。域随机化、域适应、系统辨识、元学习等技术的组合使用可以显著提高迁移成功率。特别是，大规模域随机化和感知-动作联合学习已经在高速敏捷飞行等任务中实现了从仿真到现实的成功迁移。然而，Sim-to-Real 迁移仍然是一个开放问题，特别是在复杂室外环境、多传感器融合、长时域任务等场景下。

第五，样本效率、安全性、实时性和可解释性是当前面临的主要挑战。样本效率问题限制了 RL 在真实系统上的应用；安全性问题与航空认证要求存在冲突；实时性问题影响 RL 在资源受限平台上的部署；可解释性问题阻碍了 RL 控制器的广泛接受。与大

语言模型结合、神经辐射场应用、联邦学习、数字孪生等新兴技术为解决这些挑战提供了新的可能。

第六，标准化和开源生态的建设对于推动领域发展至关重要。目前 RL 飞行控制领域缺乏统一的评估标准、基准测试和开源数据集。建立标准化评估框架、提供高质量的仿真环境、组织公开竞赛，将有助于加速技术进步和成果转化。

展望未来，随着算法、硬件和应用场景的不断发展，强化学习有望在飞行控制领域发挥越来越重要的作用。我们期待看到更多从学术研究走向实际应用的成功案例，特别是在以下方向：(1) 安全认证的 RL 飞行控制系统；(2) 大规模商业部署的自主无人机；(3) 人机协作的智能辅助驾驶；(4) 跨平台、跨任务的通用控制策略。这些发展将推动无人机技术的进一步成熟，为人类社会创造更大价值。

参考文献

- [1] Loquercio A, Kaufmann E, Ranftl R, et al. Learning high-speed flight in the wild[J]. *Science Robotics*, 2021, 6(59): eabg5810.
- [2] Kaufmann E, Bauersfeld L, Loquercio A, et al. Champion-level drone racing using deep reinforcement learning[J]. *Nature*, 2023, 620(7976): 982-987.
- [3] Sadeghi F, Levine S. CAD2RL: Real single-image flight without a single real image[C]//*Robotics: Science and Systems*. 2017, 13: 1-10.
- [4] Koch W, Mancuso R, West R, et al. Reinforcement learning for UAV attitude control[J]. *ACM Transactions on Cyber-Physical Systems*, 2019, 3(2): 1-21.
- [5] Hwangbo J, Sa I, Siegwart R, et al. Control of a quadrotor with reinforcement learning[J]. *IEEE Robotics and Automation Letters*, 2017, 2(4): 2096-2103.
- [6] Molchanov T, Chen T, Honig W, et al. Sim-to-(multi)-real: Transfer of low-level robust control policies to multiple quadrotors[C]//*IEEE/RSJ International Conference on Intelligent Robots and Systems*. 2019: 59-66.
- [7] Tobin J, Fong R, Ray A, et al. Domain randomization for transferring deep neural networks from simulation to the real world[C]//*IEEE/RSJ International Conference on Intelligent Robots and Systems*. 2017: 23-30.
- [8] Peng X B, Andrychowicz M, Zaremba W, et al. Sim-to-real transfer of robotic control with dynamics randomization[C]//*IEEE International Conference on Robotics and Automation*. 2018: 1-8.

- [9] Finn C, Abbeel P, Levine S. Model-agnostic meta-learning for fast adaptation of deep networks[C]//International Conference on Machine Learning. 2017: 1126-1135.
- [10] Shi G, Shi X, O'Connell M, et al. Neural lander: Stable drone landing control using learned dynamics[C]//IEEE International Conference on Robotics and Automation. 2019: 9784-9790.
- [11] Bauersfeld L, Kaufmann E, Foehn P, et al. NeuroBEM: Hybrid aerodynamic quadrotor model[C]//Robotics: Science and Systems. 2021.
- [12] Song S, Naji S, Kaufmann E, et al. Flightmare: A flexible quadrotor simulator[C]//Robotics: Science and Systems. 2020.
- [13] Lin L J. Self-improving reactive agents based on reinforcement learning, planning and teaching[J]. Machine Learning, 1992, 8(3): 293-321.
- [14] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529-533.
- [15] Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms[J]. arXiv preprint arXiv:1707.06347, 2017.
- [16] Lillicrap T P, Hunt J J, Pritzel A, et al. Continuous control with deep reinforcement learning[C]//International Conference on Learning Representations. 2016.
- [17] Haarnoja T, Zhou A, Abbeel P, et al. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor[C]//International Conference on Machine Learning. 2018: 1861-1870.
- [18] Fujimoto S, Hoof H, Meger D. Addressing function approximation error in actor-critic methods[C]//International Conference on Machine Learning. 2018: 1587-1596.
- [19] Kalweit G, Huegle M, Werling M, et al. Composite branched deep reinforcement learning for robotic manipulation tasks[C]//IEEE/RSJ International Conference on Intelligent Robots and Systems. 2020: 9261-9267.
- [20] OpenAI, Akkaya I, Andrychowicz M, et al. Solving rubik's cube with a robot hand[J]. arXiv preprint arXiv:1910.07113, 2019.
- [21] Andrychowicz M, Baker B, Chociej M, et al. Learning dexterous in-hand manipulation[J]. The International Journal of Robotics Research, 2020, 39(1): 3-20.
- [22] Lee J, Hwangbo J, Wellhausen L, et al. Learning quadrupedal locomotion over challenging terrain[J]. Science Robotics, 2020, 5(47): eabc5986.

- [23] Miki T, Lee J, Hwangbo J, et al. Learning robust perceptive locomotion for quadrupedal robots in the wild[J]. *Science Robotics*, 2022, 7(62): eabk2822.
- [24] Zhou A, Kumar V. Learning quadrupedal locomotion with model-based reinforcement learning and trajectory optimization[C]//*Conference on Robot Learning*. 2022: 239-249.
- [25] O’Connell M, Shi G, Shi X, et al. Neural-fly enables rapid learning for agile flight in strong winds[J]. *Science Robotics*, 2022, 7(66): eabm6597.
- [26] Romero A, Penicka R, Kaufmann E, et al. Model-based prioritized experience replay for efficient robot learning[C]//*IEEE International Conference on Robotics and Automation*. 2022: 4992-4998.
- [27] Panerati J, Zheng H, Zhou S, et al. Learning to fly: a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control[C]//*IEEE/RSJ International Conference on Intelligent Robots and Systems*. 2021: 7512-7519.
- [28] Foehn P, Kaufmann E, Romero A, et al. Crazyswarm 2: A modular quadrotor simulator[C]//*IEEE International Conference on Robotics and Automation*. 2022: 10520-10526.
- [29] Antonova R, Varava A, Orthey A, et al. Dynamic grasping with reachability and motion awareness[C]//*IEEE/RSJ International Conference on Intelligent Robots and Systems*. 2021: 2513-2520.
- [30] Haarnoja T, Moran B, Lever G, et al. Learning agile locomotion skills with a mentor[C]//*IEEE International Conference on Robotics and Automation*. 2019: 672-678.
- [31] Ibarz J, Tan J, Finn C, et al. How to train your robot with deep reinforcement learning: lessons from benchmarked robotic learning[J]. *Current Opinion in Electrochemistry*, 2021, 13(2): 132-144.
- [32] Silver D, Huang A, Maddison C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. *Nature*, 2016, 529(7587): 484-489.
- [33] Silver D, Schrittwieser J, Simonyan K, et al. Mastering the game of go without human knowledge[J]. *Nature*, 2017, 550(7676): 354-359.
- [34] Mnih V, Kavukcuoglu K, Silver D, et al. Playing atari with deep reinforcement learning[C]//*NeurIPS Deep Learning Workshop*. 2013.
- [35] Schulman J, Levine S, Abbeel P, et al. Trust region policy optimization[C]//*International Conference on Machine Learning*. 2015: 1889-1897.

- [36] Watkins C J, Dayan P. Q-learning[J]. Machine Learning, 1992, 8(3-4): 279-292.
- [37] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double q-learning[C]//AAAI Conference on Artificial Intelligence. 2016, 30(1).
- [38] Wang Z, Schaul T, Hessel M, et al. Dueling network architectures for deep reinforcement learning[C]//International Conference on Machine Learning. 2016: 1995-2003.
- [39] Mnih V, Badia A P, Mirza M, et al. Asynchronous methods for deep reinforcement learning[C]//International Conference on Machine Learning. 2016: 1928-1937.
- [40] Brockman G, Cheung V, Pettersson L, et al. OpenAI gym[J]. arXiv preprint arXiv:1606.01540, 2016.
- [41] Gu S, Lillicrap T, Sutskever I, et al. Continuous deep q-learning with model-based acceleration[C]//International Conference on Machine Learning. 2016: 2829-2838.
- [42] Heess N, Wayne G, Silver D, et al. Learning continuous control policies by stochastic value gradients[C]//Advances in Neural Information Processing Systems. 2015: 2944-2952.
- [43] Watter M, Springenberg J, Boedecker J, et al. Embed to control: A locally linear latent dynamics model for control from raw images[C]//Advances in Neural Information Processing Systems. 2015: 2746-2754.
- [44] Levine S, Finn C, Darrell T, et al. End-to-end training of deep visuomotor policies[J]. The Journal of Machine Learning Research, 2016, 17(1): 1334-1373.
- [45] Gu S, Holly E, Lillicrap T, et al. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates[C]//IEEE International Conference on Robotics and Automation. 2017: 3389-3396.
- [46] Kalashnikov D, Irpan A, Pastor P, et al. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation[J]. arXiv preprint arXiv:1806.10293, 2018.
- [47] Vecerik M, Hester T, Scholz J, et al. Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards[J]. arXiv preprint arXiv:1707.08817, 2017.
- [48] Nair A, McGrew B, Andrychowicz M, et al. Overcoming exploration in reinforcement learning with demonstrations[C]//IEEE International Conference on Robotics and Automation. 2018: 6292-6299.

- [49] Rajeswaran A, Ghotra S, Ravindran B, et al. Eptot: Learning robust neural network policies using model ensembles[C]//International Conference on Learning Representations. 2017.
- [50] Pinto L, Davidson J, Sukthankar R, et al. Robust adversarial reinforcement learning[C]//International Conference on Machine Learning. 2017: 2817-2826.
- [51] Tzeng E, Devin C, Duan Y, et al. Adapting deep visuomotor representations with weak pairwise constraints[C]//Workshop on the Algorithmic Foundations of Robotics. 2016.
- [52] Tzeng E, Hoffman J, Zhang N, et al. Deep domain confusion: Maximizing for domain invariance[J]. arXiv preprint arXiv:1412.3474, 2014.
- [53] Bousmalis K, Silberman N, Dohan D, et al. Unsupervised pixel-level domain adaptation with generative adversarial networks[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2017: 3722-3731.
- [54] Ganin Y, Lempitsky V. Unsupervised domain adaptation by backpropagation[C]//International Conference on Machine Learning. 2015: 1180-1189.
- [55] Long M, Cao Y, Wang J, et al. Learning transferable features with deep adaptation networks[C]//International Conference on Machine Learning. 2015: 97-105.
- [56] Hoffman J, Tzeng E, Park T, et al. Fcns in the wild: Pixel-level adversarial and constraint-based adaptation[J]. arXiv preprint arXiv:1612.02649, 2016.
- [57] Amodei D, Olah C, Steinhardt J, et al. Concrete problems in AI safety[J]. arXiv preprint arXiv:1606.06565, 2016.
- [58] Garcia J, Fernández F. A comprehensive survey on safe reinforcement learning[J]. Journal of Machine Learning Research, 2015, 16(1): 1437-1480.
- [59] Achiam J, Held D, Tamar A, et al. Constrained policy optimization[C]//International Conference on Machine Learning. 2017: 22-31.
- [60] Berkenkamp F, Turchetta M, Schoellig A, et al. Safe model-based reinforcement learning with stability guarantees[C]//Advances in Neural Information Processing Systems. 2017: 908-918.
- [61] Fisac J F, Akametalu A K, Zeilinger M N, et al. A general safety framework for learning-based control in uncertain robotic systems[J]. IEEE Transactions on Automatic Control, 2018, 64(7): 2737-2752.

- [62] Chow Y, Nachum O, Duenez-Guzman E, et al. A lyapunov-based approach to safe reinforcement learning[C]//Advances in Neural Information Processing Systems. 2018: 8095-8104.
- [63] Thananjeyan B, Balakrishna A, Nair S, et al. Recovery rl: Safe reinforcement learning with learned recovery zones[J]. IEEE Robotics and Automation Letters, 2021, 6(3): 4915-4922.
- [64] McGrew B. Sim-to-real transfer with neural network policies[J]. OpenAI Blog, 2019.
- [65] OpenAI. Learning dexterous in-hand manipulation[J]. arXiv preprint arXiv:1808.00177, 2018.
- [66] Rusu A A, Vecerik M, Rothörl T, et al. Sim-to-real robot learning from pixels with progressive nets[C]//Conference on Robot Learning. 2017: 262-270.
- [67] James S, Wohlhart P, Kalakrishnan M, et al. Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks[C]//IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 12627-12637.
- [68] Tan J, Zhang T, Coumans E, et al. Sim-to-real: Learning agile locomotion for quadruped robots[C]//Robotics: Science and Systems. 2018.
- [69] Hwangbo J, Lee J, Dosovitskiy A, et al. Learning agile and dynamic motor skills for legged robots[J]. Science Robotics, 2019, 4(26): eaau5872.
- [70] Loquercio A, Maqueda A I, Del-Blanco C R, et al. DroNet: Learning to fly by driving[J]. IEEE Robotics and Automation Letters, 2018, 3(2): 1088-1095.
- [71] Smolyanskiy N, Kamenev A, Smith J, et al. Toward low-flying autonomous mav trail navigation using deep neural networks for environmental awareness[C]//IEEE/RSJ International Conference on Intelligent Robots and Systems. 2017: 4241-4247.
- [72] Giusti A, Guzzi J, Ciresan D C, et al. A machine learning approach to visual perception of forest trails for mobile robots[J]. IEEE Robotics and Automation Letters, 2016, 1(2): 661-667.
- [73] Kaufmann E, Gehrig M, Foehn P, et al. Beauty and the beast: Optimal methods meet learning for drone racing[C]//IEEE International Conference on Robotics and Automation. 2019: 690-696.
- [74] Foehn P, Romero A, Kaufmann E, et al. Alphapilot: Autonomous drone racing[J]. Autonomous Robots, 2021: 1-14.

- [75] Moon H, Martinez-Carranza J, Cieslewski T, et al. Challenging environments for vision-based state estimation: The icra 2021 mohave uav racing competition[J]. IEEE Robotics and Automation Letters, 2021, 6(4): 8019-8026.
- [76] Roscia M, Foehn P, Kaufmann E, et al. Learning to fly in seconds[J]. IEEE Robotics and Automation Letters, 2023.
- [77] Song Y, Romero A, Muller M, et al. Flight goggles: Photorealistic sensor simulation for perception-driven robotics using photogrammetry and virtual reality[C]//IEEE/RSJ International Conference on Intelligent Robots and Systems. 2019: 6941-6948.
- [78] Lin X, van Kampen E J, Chu Q P. Soft actor-critic for flight control[C]//AIAA Scitech 2020 Forum. 2020: 0996.
- [79] Shi G, Shi X, O'Connell M, et al. Neural lander: Stable drone landing control using learned dynamics[C]//IEEE International Conference on Robotics and Automation. 2019: 9784-9790.
- [80] Kaufmann E, Bauersfeld L, Loquercio A, et al. Benchmarking deep reinforcement learning for vision-based agile quadrotor control[C]//Robotics: Science and Systems. 2022.
- [81] Baiza J C, Ryou G, Tal E, et al. Agile but safe: Learning collision avoidance for high-speed flight[C]//IEEE International Conference on Robotics and Automation. 2023: 6533-6539.
- [82] Aström K J, Hägglund T. Advanced PID control[M]. ISA-The Instrumentation, Systems, and Automation Society, 2006.
- [83] Aström K J, Wittenmark B. Adaptive control[M]. Courier Corporation, 2013.
- [84] Bacon P L, Harb J, Precup D. The option-critic architecture[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2017, 31(1).
- [85] Bellman R. Dynamic programming[M]. Princeton University Press, 1957.
- [86] Bengio Y, Louradour J, Collobert R, et al. Curriculum learning[C]//Proceedings of the 26th Annual International Conference on Machine Learning. 2009: 41-48.
- [87] Coumans E, Bai Y. Pybullet, a python module for physics simulation for games, robotics and machine learning[J]. GitHub Repository, 2016.

- [88] Ganin Y, Ustinova E, Ajakan H, et al. Domain-adversarial training of neural networks[J]. The Journal of Machine Learning Research, 2016, 17(1): 2096-2030.
- [89] Ho J, Ermon S. Generative adversarial imitation learning[C]//Advances in Neural Information Processing Systems. 2016: 4565-4573.
- [90] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural Computation, 1997, 9(8): 1735-1780.
- [91] Janner M, Fu J, Zhang M, et al. When to trust your model: Model-based policy optimization[C]//Advances in Neural Information Processing Systems. 2019: 12519-12530.
- [92] Koenig N, Howard A. Design and use paradigms for gazebo, an open-source multi-robot simulator[C]//IEEE/RSJ International Conference on Intelligent Robots and Systems. 2004, 3: 2149-2154.
- [93] Konda V R, Tsitsiklis J N. Actor-critic algorithms[C]//Advances in Neural Information Processing Systems. 2000: 1008-1014.
- [94] Kumar A, Zhou A, Tucker G, et al. Conservative q-learning for offline reinforcement learning[C]//Advances in Neural Information Processing Systems. 2020, 33: 1179-1191.
- [95] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [96] LeCun Y, Bengio Y, Hinton G. Deep learning[J]. Nature, 2015, 521(7553): 436-444.
- [97] Ljung L. System identification: Theory for the user[M]. Prentice Hall, 1999.
- [98] Lowe R, Wu Y, Tamar A, et al. Multi-agent actor-critic for mixed cooperative-competitive environments[C]//Advances in Neural Information Processing Systems. 2017: 6379-6390.
- [99] Mayne D Q, Rawlings J B, Rao C V, et al. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789-814.
- [100] Pan S J, Yang Q. A survey on transfer learning[J]. IEEE Transactions on Knowledge and Data Engineering, 2009, 22(10): 1345-1359.
- [101] Pomerleau D A. Efficient training of artificial neural networks for autonomous navigation[J]. Neural Computation, 1991, 3(1): 88-97.

- [102] Puterman M L. Markov decision processes: Discrete stochastic dynamic programming[M]. John Wiley & Sons, 2014.
- [103] Shah S, Dey D, Lovett C, et al. Airsim: High-fidelity visual and physical simulation for autonomous vehicles[C]//Field and Service Robotics. 2018: 621-635.
- [104] Stengel R F. Optimal control and estimation[M]. Courier Corporation, 1994.
- [105] Sutton R S, McAllester D, Singh S, et al. Policy gradient methods for reinforcement learning with function approximation[C]//Advances in Neural Information Processing Systems. 2000: 1057-1063.
- [106] Williams R J. Simple statistical gradient-following algorithms for connectionist reinforcement learning[J]. Machine Learning, 1992, 8(3): 229-256.

A RL 算法超参数设置

在实际应用中，RL 算法的超参数设置对性能有显著影响。本附录总结了飞行控制任务中常用算法的典型超参数设置。

A.1 PPO 超参数

PPO 是飞行控制中最常用的算法之一，其主要超参数包括：

- 学习率： 3×10^{-4} ，通常使用 Adam 优化器
- 折扣因子 γ ：0.99，用于计算累积回报
- GAE 参数 λ ：0.95，用于优势估计
- 裁剪参数 ϵ ：0.2，限制策略更新幅度
- 每次迭代的训练轮数：10-20 轮
- 批量大小：2048-4096 个时间步
- 价值函数损失系数：0.5
- 熵正则化系数：0.01，鼓励探索

A.2 SAC 超参数

SAC 因其样本效率高而在飞行控制中受到青睐：

- Actor 学习率： 3×10^{-4}
- Critic 学习率： 3×10^{-4}
- 折扣因子 γ ：0.99
- 目标网络软更新系数 τ ：0.005
- 回放缓冲区大小： 10^6
- 批次大小：256
- 温度参数 α ：自动调整或固定为 0.2
- 目标熵： $-\dim(A)$ ，即动作维度的负数

A.3 DDPG/TD3 超参数

DDPG 和 TD3 适用于连续控制任务：

- Actor 学习率： 10^{-3}
- Critic 学习率： 10^{-3}
- 折扣因子 γ ：0.99
- 目标网络软更新系数 τ ：0.005
- 回放缓冲区大小： 10^6
- 批次大小：100-256
- Ornstein-Uhlenbeck 噪声参数： $\theta = 0.15, \sigma = 0.2$
- 策略网络更新频率：每 2 次 Critic 更新更新 1 次 Actor(TD3)

B 仿真环境配置

B.1 Gazebo 配置

Gazebo 是 ROS 生态中最常用的仿真器，配置步骤如下：

1. 安装 ROS 和 Gazebo: `sudo apt-get install ros-noetic-desktop-full`

2. 安装飞行插件: `sudo apt-get install ros-noetic-rotors-simulator`
3. 配置世界文件, 设置重力、物理引擎 (ODE/Bullet/Simbody)
4. 配置飞行器 URDF/SDF 模型文件
5. 启动仿真: `roslaunch rotors-gazebo mav-hovering-example.launch`

B.2 AirSim 配置

AirSim 提供高保真的视觉渲染, 配置步骤:

1. 下载 Unreal Engine 和 AirSim 源码
2. 编译 AirSim: `build.cmd` (Windows) 或 `build.sh` (Linux)
3. 配置 `settings.json` 文件, 设置飞行器型号、传感器配置
4. 启动 Unreal 环境, AirSim 自动作为插件加载
5. 使用 Python API 连接: `client = airsims.MulticopterClient()`

B.3 PyBullet 配置

PyBullet 适合快速原型开发和强化学习训练:

```
import pybullet as p
import pybullet_data

# 连接到仿真器
physicsClient = p.connect(p.GUI)

# 加载飞行器模型
p.setAdditionalSearchPath(pybullet_data.getDataPath())
planeId = p.loadURDF("plane.urdf")
droneId = p.loadURDF("quadrotor.urdf", [0, 0, 1])

# 设置仿真参数
p.setGravity(0, 0, -9.81)
p.setTimeStep(1./240.)

# 运行仿真
for i in range(10000):
    p.stepSimulation()
```

C 奖励函数设计模板

C.1 姿态控制奖励

$$r = -w_1 \|\mathbf{q}_{error}\|^2 - w_2 \|\boldsymbol{\omega}\|^2 - w_3 \|\mathbf{a}\|^2 - w_4 \|\dot{\mathbf{a}}\|^2 \quad (40)$$

其中 $\mathbf{q}_{error}$ 是姿态误差四元数, $\boldsymbol{\omega}$ 是角速度, $\mathbf{a}$ 是动作, $\dot{\mathbf{a}}$ 是动作变化率。

典型权重值: $w_1 = 1.0$, $w_2 = 0.01$, $w_3 = 0.001$, $w_4 = 0.0001$

C.2 位置跟踪奖励

$$r = -w_1 \|\mathbf{p} - \mathbf{p}_{target}\|^2 - w_2 \|\mathbf{v} - \mathbf{v}_{target}\|^2 - w_3 \|\mathbf{a}\|^2 + w_4 r_{success} \quad (41)$$

其中 $\mathbf{p}$ 是位置, $\mathbf{v}$ 是速度, $r_{success}$ 是任务完成奖励。

典型权重值: $w_1 = 1.0$, $w_2 = 0.1$, $w_3 = 0.001$, $w_4 = 10.0$

C.3 着陆奖励

着陆是飞行中最关键的阶段之一, 奖励函数需要综合考虑多个因素:

$$r = r_{touchdown} + r_{sink_rate} + r_{position} + r_{attitude} + r_{safety} \quad (42)$$

各项奖励定义:

- $r_{touchdown}$: 正确接地时机奖励。在理想的接地时机 (高度接近 0、垂直速度适当) 给予正奖励, 过早或过晚接地给予负奖励。
- r_{sink_rate} : 适当下沉率奖励。鼓励保持适当的下沉率 (通常 0.5-1.5 m/s), 过大或过小的下沉率都给予惩罚。
- $r_{position}$: 跑道中心线对齐奖励。鼓励飞行器对准跑道中心线, 偏离中心线越远惩罚越大。
- $r_{attitude}$: 合适姿态奖励。鼓励保持适当的俯仰角 (固定翼通常需要 flare 机动减小下沉率)、零滚转角和零偏航角。
- r_{safety} : 安全约束惩罚 (负值)。违反安全约束 (如失速、过度倾斜、过低高度) 时给予大额惩罚。

C.4 避障奖励

对于自主避障任务，奖励函数设计如下：

$$r = r_{goal} + r_{collision} + r_{clearance} + r_{smooth} \quad (43)$$

各项奖励定义：

- r_{goal} ：到达目标奖励。成功到达目标位置给予大额正奖励。
- $r_{collision}$ ：碰撞惩罚。与障碍物碰撞给予大额负奖励，终止当前 episode。
- $r_{clearance}$ ：安全距离奖励。鼓励与障碍物保持安全距离，距离越近惩罚越大。
- r_{smooth} ：轨迹平滑奖励。惩罚剧烈的动作变化，鼓励平滑的飞行轨迹。

C.5 能量最优奖励

对于长航时任务，可以设计能量最优奖励函数：

$$r = -w_1 P_{total} - w_2 \|\mathbf{a}\|^2 + w_3 v_{ground} \quad (44)$$

其中 P_{total} 是总功耗（包括推进功率、航电功耗等）， v_{ground} 是地速。该奖励函数鼓励最小化能耗同时保持适当的前进速度。

D 常用数据集与基准

D.1 飞行数据集

Agile Autonomy Dataset：ETH Zurich 发布的高速飞行数据集，包含超过 50,000 个专家飞行轨迹，速度高达 100km/h。数据集包含高分辨率图像、IMU 数据、控制指令等，适用于模仿学习和视觉导航研究。

UZH-FPV Drone Racing Dataset：包含第一人称视角 (FPV) 无人机竞速数据，包含同步的图像、IMU、控制指令和精确的位置真值。适用于高速视觉导航研究。

EuRoC MAV Dataset：包含微型飞行器在室内环境的飞行数据，包含双目图像、IMU、控制指令和精确的运动捕捉数据。适用于视觉-惯性里程计研究。

D.2 仿真基准环境

Gym-PyBullet-Drones：基于 PyBullet 的多旋翼仿真环境，支持多种飞行器模型和任务场景。提供标准化的 Gym 接口，易于与 RL 算法集成。

Flightmare：高保真度的飞行仿真器，支持视觉渲染和物理仿真。使用模块化设计，支持快速原型开发和大规模并行训练。

AirSim: 基于 Unreal Engine 的高保真仿真器, 提供逼真的视觉渲染和物理仿真。支持多平台 (多旋翼、固定翼、汽车) 和多种传感器配置。

E 术语表

表 6: 常用术语表

术语	定义
MDP	马尔可夫决策过程
RL	强化学习
DRL	深度强化学习
DQN	深度 Q 网络
DDPG	深度确定性策略梯度
TD3	双延迟深度确定性策略梯度
SAC	软演员-评论家 [17] 算法
PPO	近端策略优化
TRPO	信任区域策略优化
MAML	模型无关元学习
Sim-to-Real	从仿真到现实的迁移
DR	域随机化
DA	域适应
CBF	控制屏障函数
VTOL	垂直起降
UAV	无人机
MAV	微型飞行器
FPV	第一人称视角
IMU	惯性测量单元
GPS	全球定位系统
SLAM	同步定位与地图构建
NeRF	神经辐射场
LLM	大语言模型